



**TRANSPORTA
UN SAKARU
INSTITŪTS**

DEFENCE PERMITTED

Dean of Engineering Faculty
Professor, Ph.D.
Emmanuel Alejandro Merchan Cruz

202__ . year ____ . _____

BACHELOR THESIS

**"REAL-TIME HUMAN'S HAND MOVEMENT TRACKING ON THE BASE OF
VIDEO DATA PROCESSING"**

Computer Science, 43483

Student: **Daniils Grammatikopulo**
(st. c. 79101)

(signature) 202__ . year ____ . _____

Supervisor: *Professor, Dr. sc. ing.*
Aleksandrs Grakovskis

(signature) 202__ . year ____ . _____

Riga 2025

CONFIRMATION

I, Grammatikopulo Daniils, confirm that I have developed the given graduation thesis individually for obtaining the degree of Computer Science. By my signature I verify that the given graduation thesis is written independently and there are no violations regarding the rights of intellectual property of other persons or plagiarism. The used papers and data sources of other authors are indicated in references. The text of the submitted paper has never been submitted partially or fully to other commission for theses evaluation. I confirm that the electronic version of the submitted thesis meets completely the text of the submitted paper copy of the thesis.

(signature)

(name, surname)

CONSENT

I, Grammatikopulo Daniils, hereby agree that my graduation thesis abstract can be used by AS "TRANSPORTA UN SAKARU INSTITŪTS" (hereinafter – the Institute), reg. No. 40003458903, Lauvas street 2, Riga, LV-1019, in study and scientific work, for improving and optimizing the quality of the study process.

I hereby agree that the graduation thesis abstract will be publicly available on the Institute's website www.tsi.lv, in the Institute's Library or in academic units.

(signature)

(name, surname)

ANOTĀCIJA

Bakalaura darbs „Reālā laika cilvēka roku kustības izsekošana, pamatojoties uz video datu apstrādi”. Darba autors: Grammatikopulo Daniils. Zinātniskais vadītājs: Dr. sc. ing., profesors Aleksandrs Grakovskis.

Darbs „Datorzinātnes” grāda iegūšanai: 49 lpp., 28 att., 4 tab., 21 izmantotās lit. avot., 1 piel.

DATORVĪZIJA, DZIĻUMA NOVĒRTĒŠANA, ROKU IZSEKOŠANA, CILVĒKA UN DATORA MIJIEDARBĪBA, ŽESTU ATPAZĪŠANA

Šajā bakalaura darbā ir aprakstīta reāllaika rokas izsekošanas sistēmas izstrāde attālinātai paraksta veidošanai, izmantojot vienu kameru. Galvenais mērķis ir ļaut lietotājiem attālināti parakstīt dokumentus bez fiziskas klātbūtnes vai specializēta aprīkojuma. Sistēma izmanto datorredzes un iepriekš apmācītu modeli, izmantojot OpenCV video apstrādei un MediaPipe ietvaru rokas orientieru noteikšanai un žestu atpazīšanai. Grafiskā lietotāja saskarne ir veidota, izmantojot PySide6, lai nodrošinātu lietotājam draudzīgu vidi konfigurēšanai un mijiedarbībai. Risinājums ļauj lietotājiem kontrolēt zīmēšanas procesu, izmantojot tādus žestus kā “rādīt uz augšu”, lai zīmētu, un “īkšķi uz augšu/uz leju”, lai saglabātu vai notīrītu audeklu. Sistēma ir testēta, tostarp veikti eksperimenti ar kadru apstrādes optimizāciju un dziļuma novērtēšanas izpēti ar MiDaS modeli. Eksperimentālā analīze bija vērsta uz veikspējas optimizēšanu, izmantojot kadru izlaišanu, un dziļuma novērtēšanas izpēti, izmantojot MiDaS, ar ieteikumiem turpmākajiem uzlabojumiem, kas ietver fona noņemšanu un uzlabotos modeļus, piemēram, DepthAnything. Sistēma nodrošina pieejamu un viegli pieejamu alternatīvu fiziskai parakstīšanai, kas ir īpaši noderīga attālinātam darbam un situācijās, kad tieša saskarsme ir ierobežota.

ABSTRACT

Bachelor Thesis „Real-Time Human's Hand Movement Tracking on the Base of Video Data Processing”. Author of the paper: Grammatikopulo Daniils. Scientific supervisor: Dr. sc. ing., professor Aleksandrs Grakovskis.

Paper for obtaining the degree „Computer Science”: 49 pp., 28 img., 4 tabl., 21 ref., 1 app.

COMPUTER VISION, DEPTH ESTIMATION, HAND TRACKING, HUMAN COMPUTER INTERACTION, GESTURE RECOGNITION

This bachelor thesis presents the development of a real-time hand tracking system for remote signature creation using a single camera. The main goal is to allow users to sign documents remotely without the need of physical presence or specialized equipment. The system leverages computer vision and a pre-trained model, using OpenCV for video processing and MediaPipe framework for hand landmark detection and gesture recognition. A graphical user interface is built with PySide6 to provide a user-friendly environment for configuration and interaction. The solution allows users to control the drawing process via gestures like “pointing up” to draw and “thumbs up/down” to save or clear the canvas. The system has been tested, including experiments on optimizing frame processing and exploring depth estimation with the MiDaS model. Experimental analysis focused on optimizing performance through frame skipping and explored depth estimation using MiDaS, with suggestions for future improvements involving background removal and advanced models like DepthAnything. The system provides an affordable and easy-to-access alternative to physical signing, which is especially useful for remote work and situations where direct contact is limited.

TABLE OF CONTENTS

ABBREVIATIONS USED IN THE PAPER.....	7
INTRODUCTION.....	8
1. THEORETICAL BACKGROUND AND RELATED WORKS	10
1.1. Foundations of Computer Vision	10
1.2. Overview of Depth Estimation Methods.....	12
1.2.1. MiDaS	12
1.2.2. MonoViT	13
1.2.3. DepthAnything	15
1.3. Overview of Tracking	16
1.3.1. Early Methods and Depth Sensor	16
1.3.2. OpenPose.....	17
1.3.3. MediaPipe Hands.....	18
1.4. Overview of Existing Solutions	18
1.4.1. Finger Air Signing	18
1.4.2. Remote Scan Recognition	19
2. SOFTWARE DESIGN AND IMPLEMENTATION.....	20
2.1. System Analysis	20
2.1.1. Functional Requirements.....	20
2.1.2. Non-functional Requirements	21
2.1.3. System Requirements	22
2.1.4. User Requirements	23
2.1.5. Use Case Diagram	23
2.2. System Design	24
2.2.1. Sequence Diagram.....	25
2.2.2. Class Diagram	29
2.2.3. Component Diagram	29
2.3. Software Implementation.....	31
2.3.1. Wireframe	31
2.3.2. Graphical User Interface.....	32
2.3.3. Software Packaging and Installer Creation	33
2.4. Software Testing	34
2.4.1. Manual Testing	34
2.4.2. Unit Testing	38
3. EXPERIMENTAL ANALYSIS AND FUTURE IMPROVEMENTS	39
3.1. Frame Count Optimization for Neural Network Input.....	39

3.1.1. Impact of Frame Skip on Performance and Accuracy	39
3.1.2. Trajectory Visualization.....	40
3.2. Background Removal.....	42
3.2.1. Algorithm for Background Removal	42
3.2.2. Applying MiDaS.....	43
CONCLUSION	45
REFERENCE LIST	47
APPENDICES.....	49

ABBREVIATIONS USED IN THE PAPER

AR – Augmented Reality

BGR – Blue Green Red

CNN – Convolutional Neural Network

CV – Computer Vision

DL – Deep Learning

FHD – Full High Definition

FPS – Frame Per Second

GPU – Graphics Processing Unit

GUI – Graphical User Interface

HCI – Human Computer Interaction

HSV – Hue Saturation Value

PC – Personal Computer

ResNet – Residual Network

RGB – Red Green Blue

SaaS – Software as a Service

UI – User Interface

UML – Unified Modeling Language

UX – User Experience

ViT – Vision Transformers

VR – Virtual Reality

INTRODUCTION

In the world of rapid growth of technologies, real-time video processing is becoming significantly important since applications vary from security systems to interactive media. The capability of real-time video processing not only increases efficiency but also paves the way for new opportunities and innovative solutions, such as autonomous driving, smart surveillance, and augmented reality. The immediate analysis of the video flow allows for timely detection of critical events and appropriate reactions. Moreover, in the past five years the number of academic articles devoted to topics such as human-computer interaction, real-time video processing, motion detection, gesture recognition has significantly increased, this is largely related to the pandemic (*COVID-19*) and the rapid growth of deep neural networks.

The aim of the work is to develop a program that allows users to place signatures remotely in real-time using hand tracking neural network and only one camera. This frees the user from physical presence at the place where contracts and other documents are signed. Using a single camera reduces the equipment costs and simplifies technical requirements. Moreover, this approach ensures broader accessibility, as most of the users already have built-in cameras in their device, such as laptops and smartphones. This solution is particularly relevant in situations like pandemics, where physical interaction is restricted. Therefore, it is essential to be prepared. Furthermore, remote interaction has become common, making such technology highly practical.

To achieve this goal, the following tasks must be solved: developing an image processing, implementing gesture recognition, and applying a neural network for hand tracking. The object of this study is the video processing system, while the subject focuses on hand tracking, depth estimation, and gesture recognition. The implementation of the project includes several key methods and technologies, aimed at real-time video processing, hand tracking, gesture recognition and depth estimation. These methods are fundamental for creating a system of remote signatures. The basis of the project is the processing of a video stream in real-time using one camera. For capturing and processing video frames *OpenCV* was used. Implementation with the help of *OpenCV* allows to stay with high performance and compatibility with various devices.

The next essential element of the system is hand tracking, which was performed with the help of *MediaPipe* framework by Google. The *MediaPipe* provides a pretrained model for hand detection in real-time, which detects and tracks landmarks of the hand. The model recognizes specific points on the hand, such as the fingertips and wrist, allowing the system to track the user's hand movements. This data will be used for gesture recognition and determining the position of the hand. The system will detect whether the index finger is open or closed and activates or deactivates the drawing depending on the gesture. In order to clear the virtual canvas or save the

signature, “thumbs down” and “thumbs up” gesture will be used. Furthermore, the *MediaPipe* hand detection model also provides relative information about the distance of the hand from the camera. Although *MediaPipe* provides basic depth estimation, the *MiDaS* pretrained model was also considered to explore the potential of deep learning approaches. However, it was not included in the main pipeline. The *MiDaS* model estimates the depth for each pixel in the frame, providing more detailed information about the environment. However, it requires more computational resources compared to *MediaPipe* and should be run on a graphical processing unit (*GPU*).

As a result of research, a system for remote signing using a single camera will be developed, providing acceptable accuracy and convenient user experience. Additionally, based on the analysis of current state-of-the-art methods, further improvements and suggestions will be highlighted.

The structure of the work includes three main parts and a conclusion. The first part introduces the theoretical background of computer vision and review of existing research and solutions which are applicable in context of remote signature. Namely, methods of depth estimation are explored, including *MiDaS*, *MonoViT* and *DepthAnything*, but also the methods of tracking, such as *OpenPose* and *MediaPipe*. Finally, existing analogues are provided, such as “Finger Air Signing” and “Remote Scan Recognition”, in order to identify their limitations.

The second part describes the process of development, starting from system analysis to identify the requirements such as functional, non-functional, user and system. Then follows the system design, including *UML* (Unified Modelling Language) diagrams such as sequence, class, component to build the architecture and create overall understanding of the system workflow. Then, it describes the system implementation, including wireframe development, the design and explanation of the final graphical user interface, and the packaging of the software for installer creation. Subsequently, the software is tested using both manual methods and automated unit tests.

The third part of the work is dedicated to experimental analysis of the system. It introduces the optimization of video stream processing, including frame skipping to reduce the number of frames passed to the hand detection model. It also presents the collected metrics used to find the optimal frame skip value, along with trajectory plots of finger movements. Furthermore, this part includes the algorithm of background removal based on the depth map. It is considered a preprocessing step before hand detection, aimed at helping the neural network perform better in complex scenarios by removing impact of the background and enhancing the focus on the hand.

1. THEORETICAL BACKGROUND AND RELATED WORKS

In order to develop an effective system for remote signing using a single camera, it is crucial to understand the theoretical foundation of *CV* (Computer Vision). Moreover, it is obligatory to review the existing technologies and solutions in the field of hand detection, gesture, and monocular depth estimation. This part is devoted to a comprehensive overview of key concepts and methods. It begins with the discussion of computer vision basics and continues with overview of the hand tracking and depth estimation. Further, existing systems for remote signing are analysed.

In the past ten years the number of academic articles of computer vision that includes hand tracking, gesture recognition, human computer interaction and depth estimation increased, according to the statistics from *Scopus* (see Fig. 1), demonstrating the importance of computer vision nowadays. This rapid growth highlights the demand for intelligent systems that are capable of understanding *HCI* (Human Computer Interaction) in real-time. Such technologies are widely applied in fields like *VR/AR* (Virtual Reality/Augmented Reality). Additionally, the progress in machine learning and availability of large, labelled datasets boosted research in this domain.

Documents by year

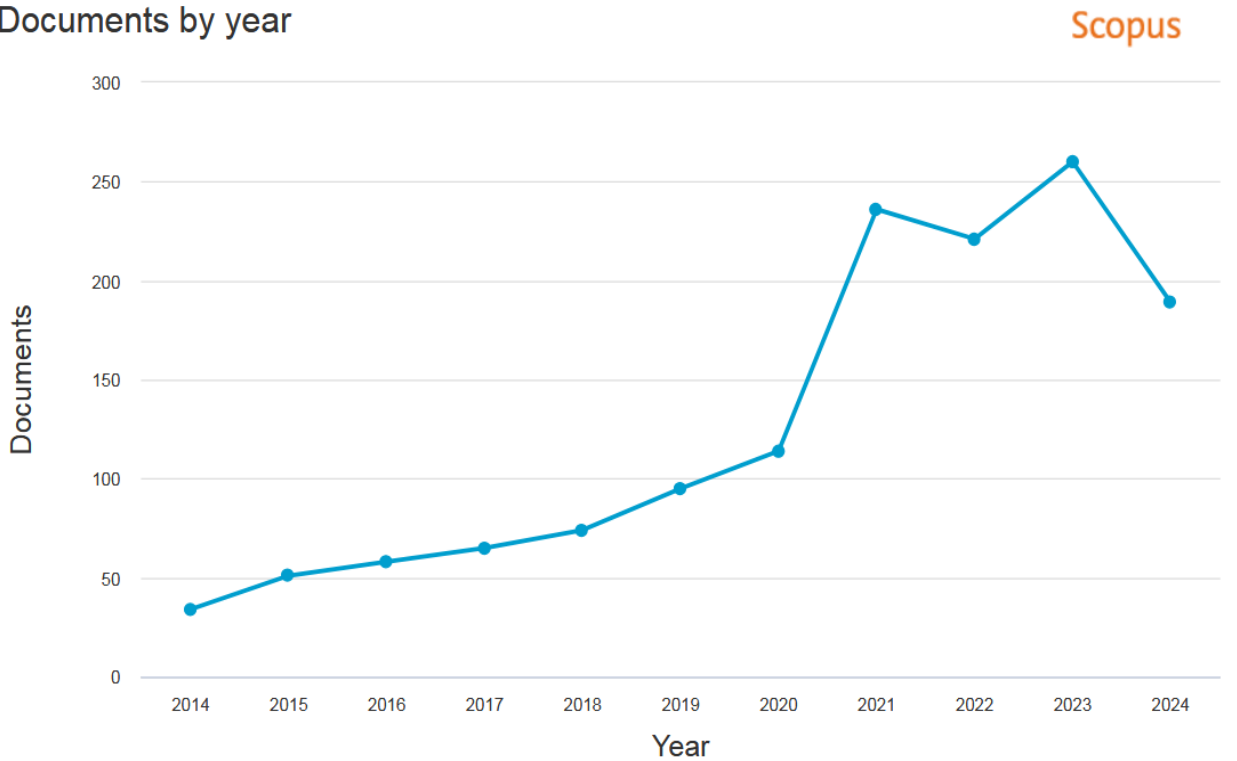


Fig. 1. Growth in publications in *Scopus*

1.1. Foundations of Computer Vision

An *image* is the two-dimensional function $f(x, y)$ that depend on two coordinates: horizontal (x) and vertical (y). Each point of this function represents intensity, meaning how light or dark this point is. If all coordinates and intensity values are discrete, the image is called a *digital image*, as

described by Gonzalez and Woods (2017). The *digital image* consists of a finite number of elements, each with specific location and value. These elements are called *pixels*. Pixels form a grid, where pixels can be represented in the following *colour models*:

- *RGB* – this model is the most common and is based on mixing three core colours: red, green, blue. Sometimes this model has the fourth channel *a* (alpha channel) to manage the transparency of the pixel.
- *BGR* – this model is the same as *RGB*, but with reversed channel order. It is used in some systems and libraries such as *OpenCV* (used in this work).
- *HSV* – this model consists of three colour components: hue, saturation, and value (brightness). This model is widely used for segmentation or detection of the object by colour.
- *Grayscale* – this model has only one channel (light intensity) and does not contain colour information, this makes this model more compact in terms of memory and is widely used for edge detectors (*Canny*, *Sobel*).

A *digital image* can also be expressed in a conventional matrix form $n \times m$, as shown below see (1). In the matrix, the column number shows the pixel's position on the X-axis (left to right), and the row shows its position on the Y-axis (top to bottom):

$$f(x, y) = \begin{bmatrix} f(0, 0) & \cdots & f(0, n-1) \\ \vdots & \ddots & \vdots \\ f(m-1, 0) & \cdots & f(m-1, n-1) \end{bmatrix} \quad (1)$$

However, it is not the clearest representation of digital image, it can be simplified by substitution. Now the matrix can be rewritten using $a_{ij} = f(i, j)$, where *a* represents the value of colour models. As a result, the following matrix is obtained (2):

$$A \in R^{n \times m} = \begin{bmatrix} a_{0,0} & \cdots & a_{0,n-1} \\ \vdots & \ddots & \vdots \\ a_{m-1,0} & \cdots & a_{m-1,n-1} \end{bmatrix} \quad (2)$$

Now this matrix can be represented in more intuitive way (see Fig. 2), where each cell corresponds to a pixel. The magnified view of pixel a_{00} shows its *RGB* colour values: red – 255, green – 255, blue – 213. These principles of digital image processing form the basis of all subsequent stages of the work – from capturing an image from a camera to its analysis using neural networks for recognizing hand and gestures.

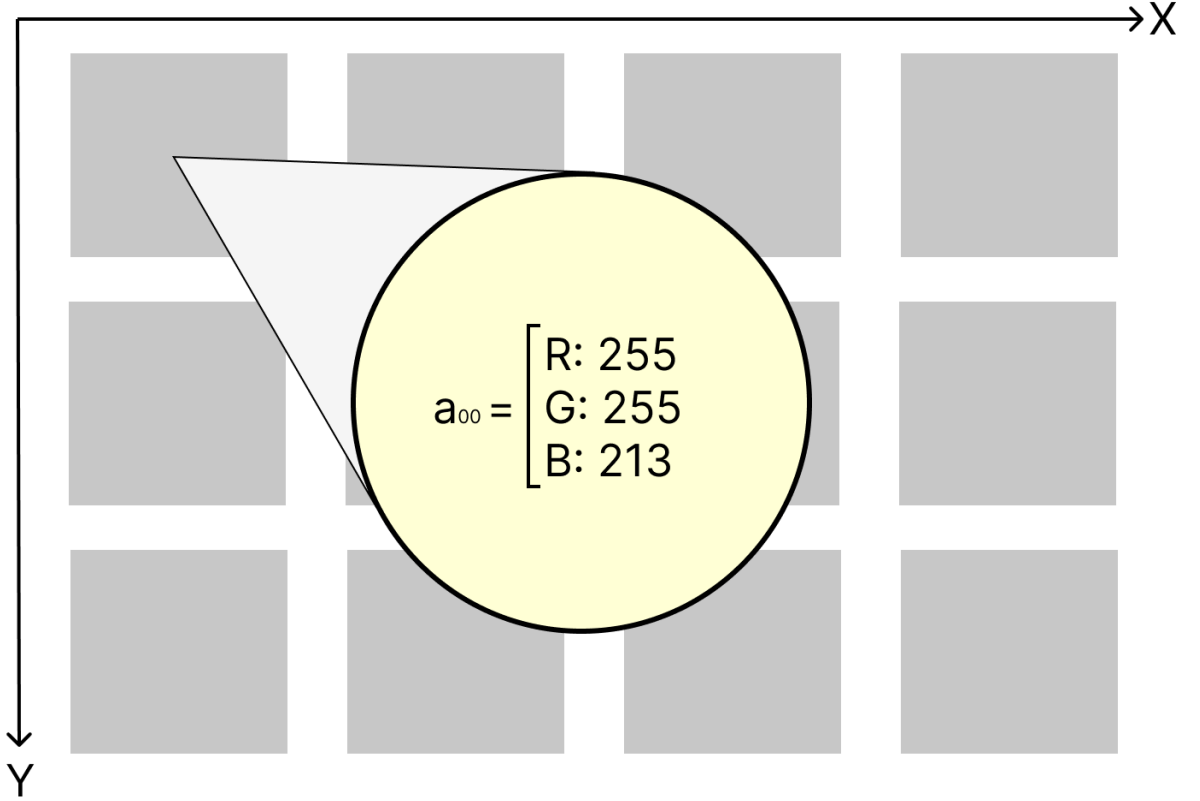


Fig. 2. Visualisation of a digital image matrix

1.2. Overview of Depth Estimation Methods

Depth estimation has become popular in recent years due to rapid development of computer vision and machine learning technologies. The task involves determining the distance to objects within an image or video stream. This is crucial for more of the application: from augmented and virtual reality to autonomous driving systems. There are various approaches to depth estimation, including methods based on stereo vision, the use of sensors, and modern methods based on neural network training. This section will be devoted to modern machine learning solutions.

1.2.1. *MiDaS*

One of the most popular neural networks models for monocular depth estimation is *MiDaS* (Ranftl *et al.*, 2022). The *MiDaS* is trained on large data sets and can predict relative depth from one RGB image with the strong generalization for diverse cases. The model uses encoder-decoder structure, where the encoder is based on the Residual Network (ResNet) architecture (He *et al.*, 2015), which helps to extract high-level features by making it easier to train deep networks using residual connections. *MiDaS* is actively used for both static images and video. However, the performance in real-time is limited due to computational complexity, which causes difficulties in

interactive applications. There are a lot of models with different architecture, here are three of the most popular and easy to use models:

- *MiDaS v2.1 Small* – a lightweight neural network.
- *MiDaS v3.0 Hybrid* – a more accurate than small version that utilizes *ResNet-50*.
- *MiDaS v3.0 Large* – the most accurate but also the slowest version that utilizes either *Vision Transformer* or *ResNeXt-101*.

The image below (see Fig. 3) demonstrates a visual comparison of different monocular depth estimation models. Newer and larger models tend to produce better results, making object edges sharper. Also, it is noticeable that very distant objects are rendered more clearly by the more modern models, however, it takes more power of computer.

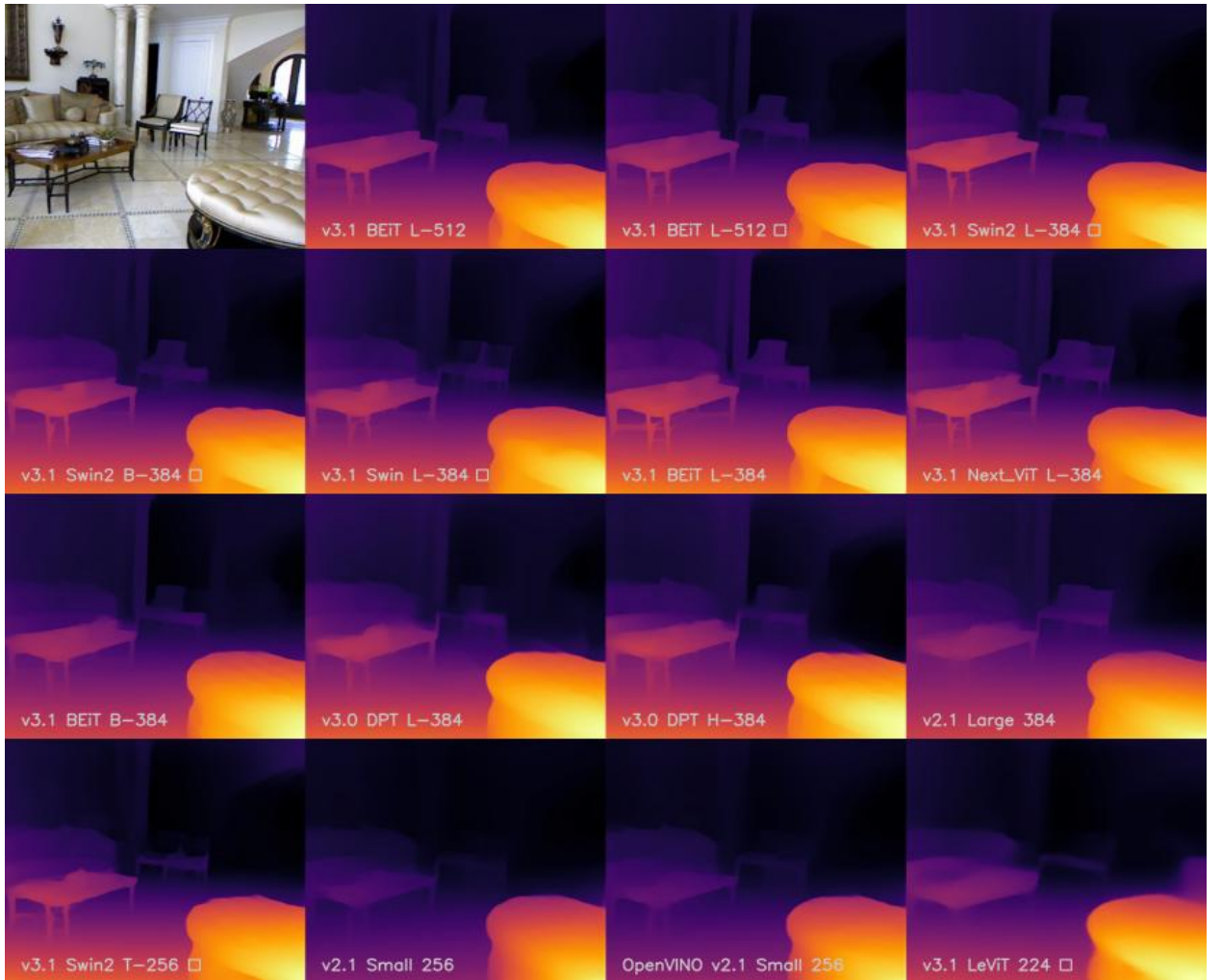


Fig. 3. Depth maps comparison generated by various *MiDaS* models (Shapiro and Malinowski, 2023)

1.2.2. MonoViT

Another modern model for depth estimation based on one image is *MonoViT*. It is a hybrid architecture, which consists of *CNN* (Convolutional Neural Networks) and Vision Transformer to combine local and global image context. The model trains in self-training mode, using reconstruction

of the adjacent frames to obtain an error signal. Thus, it allows them to work without labelled depth map. According to the test results on datasets *KITTI* and *DrivingStereo* showed high accuracy and generalization, outperforming other self-supervised approaches such as:

- *CADepth* – uses cross-attention to improve depth estimation by integrating local and global information (Yan *et al.*, 2021).
- *DIFFNet* – uses differentiable filters to optimize depth estimation, effectively handling various scene contexts (Zhou, Greenwood and Taylor, 2021).

The picture below shows (see Fig. 4) that *MonoViT* produces sharper object boundaries and avoids artifacts even under challenging or low-quality visual conditions, unlike other methods. This is especially noticeable in the 4th and 5th images: in the 4th, where the motorcyclist has clearer edges and, in the 5th, where *MonoViT* did not create an artifact on the road due to the poor quality of the image. However, *ViT* (Vision Transformer) is the reason for usage of more computational resources, which may limit its real-time application on low-end devices (Zhao *et al.*, 2022).

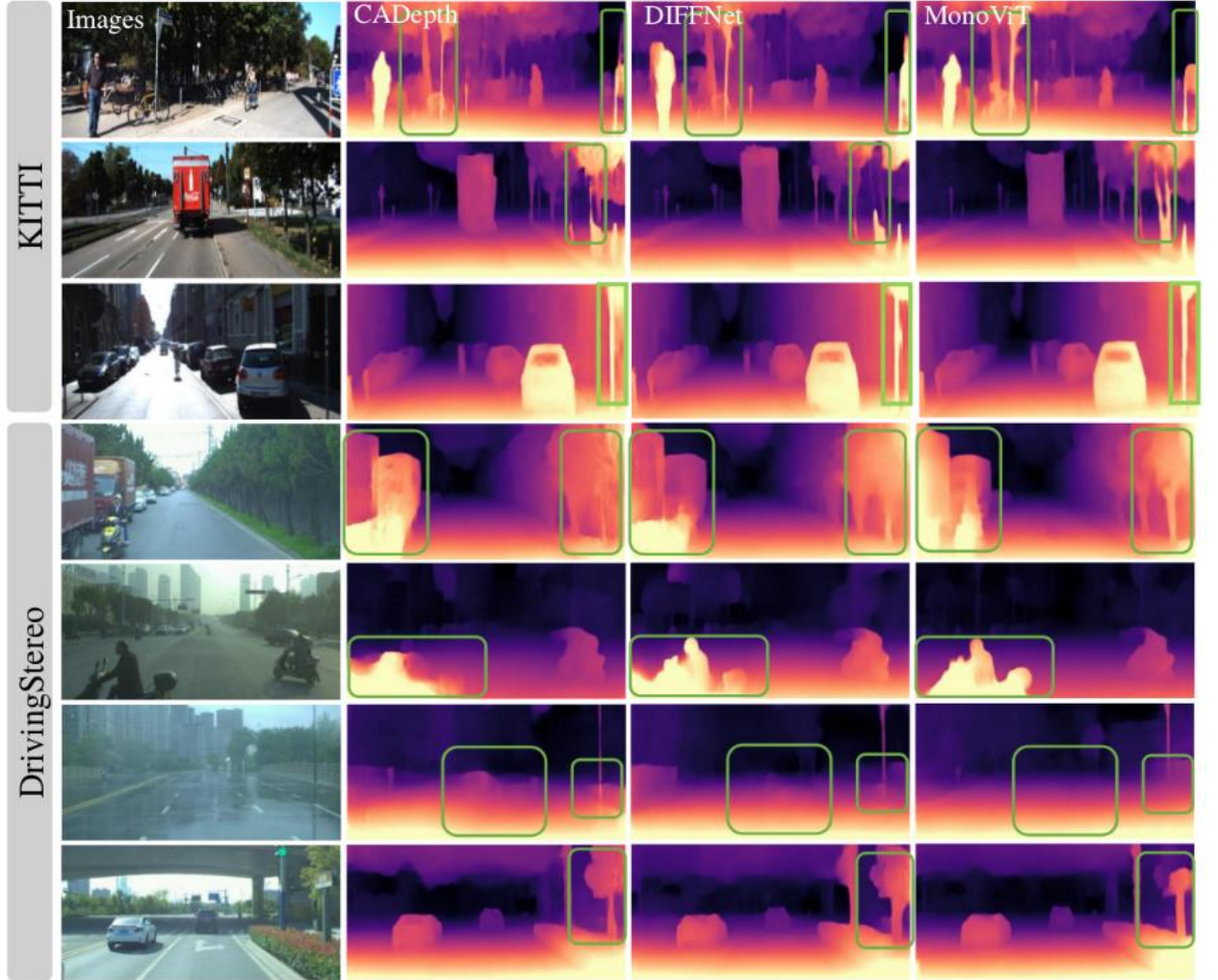


Fig. 4. Comparison of different self-supervised approaches with *MonoViT* (Zhaocq, 2022)

1.2.3. *DepthAnything*

The paper “*Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data*” by Yang et al. (2024) introduced universal model for monocular depth estimation designed with a focus on using unlabelled data at scale over 62 million high-quality images. They developed an automatic annotation system, which uses pre-trained models to generate pseudo-labels of depth. To increase the stability and generalization ability of the models the following approaches were used:

- Model is trained on images, which are manually distorted (colour change, blur, cropping etc.). This helps to predict the depth even in the complex conditions.
- To help the model “understand” the scene better, it is trained to keep the meaning of the image by checking its internal features with the “smart” encoder *DINOv2*. *DINOv2* is a self-supervised model that learns to understand images without labels (Oquab *et al.*, 2023).

As a result, experiments showed that *DepthAnything* outperformed models such as *MiDaS* (see Fig. 5) on various datasets, including *KITTI* and *NYUv2*. It can also be observed that the model predicts depth even for small details. Additionally, the model performed very well on the drawn image and was able to show significant results compared to *MiDaS*.

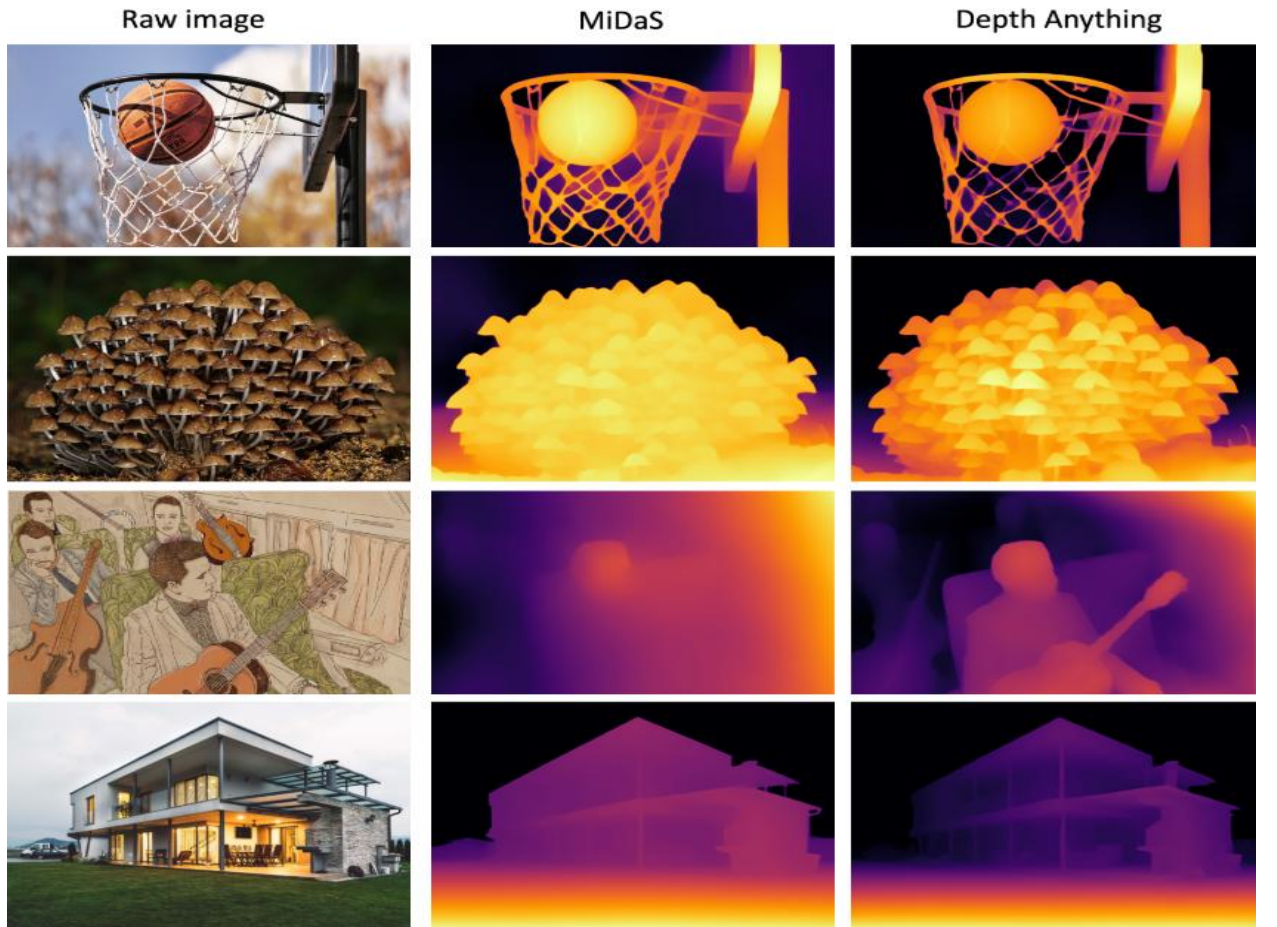


Fig. 5. *DepthAnything* vs *MiDaS* v3.1 (*Depth Anything*, 2024)

1.3. Overview of Tracking

Hand tracking is a crucial component of modern system of *HCI*, providing intuitive and contactless control interfaces. The development of hand tracking has gone through significant evolution in the past ten years. The methods ranged from simple mathematical approaches to deep neural networks. Currently, neural networks have surpassed previous solutions and are gaining more popularity, making the systems more flexible and accurate.

1.3.1. Early Methods and Depth Sensor

Initially, hand tracking was based on using gloves with sensors and markers to capture movement of the hand. This solution was quite expensive and limited in terms of application because of the necessity of physical interaction with devices (Wachs *et al.*, 2011). With the development of *CV*, new methods of detection appeared without markers using traditional algorithms of image processing, such as hand colour segmentation, background subtraction and edge analysis. However, these methods are quite sensitive to poor illumination and complexities of the background.

The next breakthrough in gesture recognition and 3D tracking was depth sensors such as *Microsoft Kinect* in 2010. It allows tracking the movement of the hand in three-dimensional space in real-time using infrared depth data. These technologies have quickly become famous in games and interactive applications (Shotton *et al.*, 2011). However, in this article researchers focused not only on hand movement, but on the whole body. They suggested idea of pose recognition without using time information (without frame sequence analysis). They split task into two parts:

- Per-pixel classification – each pixel of depth-map is classified as a body part (31 classes).
- Pose generation – from “clouds” of pixels of a certain class, they find 3D coordinates of joints using *Mean Shift*, which allows to find local maximum of the density in 3D (Comaniciu and Meer, 2002).

As a result, they successfully managed to train *Random Forest* on a large dataset of 1 million images and obtained 200 FPS (Frame Per Second) on GPU (Graphics Processing Unit) Xbox. The picture (see Fig. 6) below shows how each part of the body was classified based on the depth data. The image demonstrates the process of body part recognition using an algorithm:

- On the left are the original depth images.
- In the centre is the classification result with color-coded body parts.
- On the right are 3D joint proposals shown from different perspectives.

This visualisation of pixel classification for human pose estimation without temporal information marked a major breakthrough in computer vision.

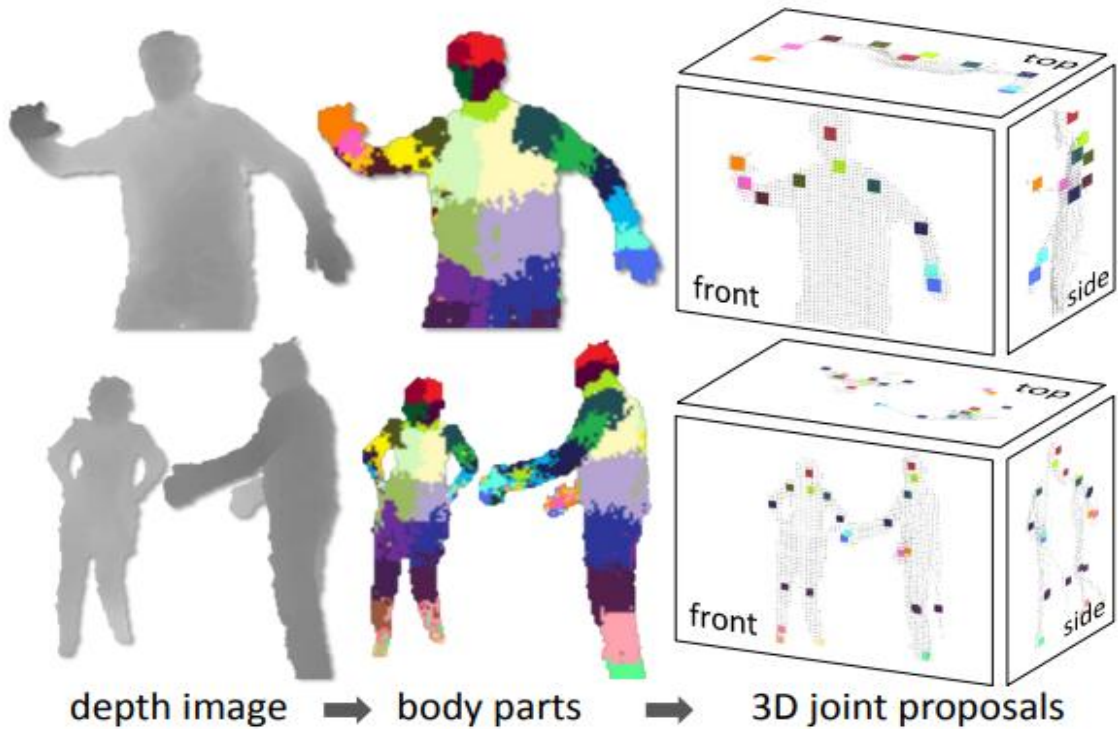


Fig. 6. Pose recognition from depth image (Shotton *et al.*, 2011)

1.3.2. *OpenPose*

The development of *DL* (Deep Learning) has significantly improved the accuracy of hand tracking systems. For instance, *OpenPose* (Cao *et al.*, 2017) suggested algorithms for detection of key points of the body and hands of the image in real-time using *CNN* to predict heatmaps and part affinity fields. Thus, it allowed to work even in complex conditions. Moreover, this solution eliminated the necessity of acquiring costly peripheral devices, such as depth cameras and others. The picture (see Fig. 7) below shows the landmarks identified by *OpenPose*.



Fig. 7. *OpenPose* tracking system (Cao *et al.*, 2017)

1.3.3. MediaPipe Hands

Based on these ideas in 2020 *Google* introduced *MediaPipe Hands* – light-weight framework. The *MediaPipe* provides localization of 21 key points of the hand (see Fig. 8) and can detect two hands simultaneously in real-time with the high accuracy. Moreover, it is able to predict the depth for each landmark of the hands, which is quicker than monocular depth estimation methods (Zhang *et al.*, 2020). Additionally, *MediaPipe* offers a gesture recognition model that operates asynchronously and supports the gesture recognition models.

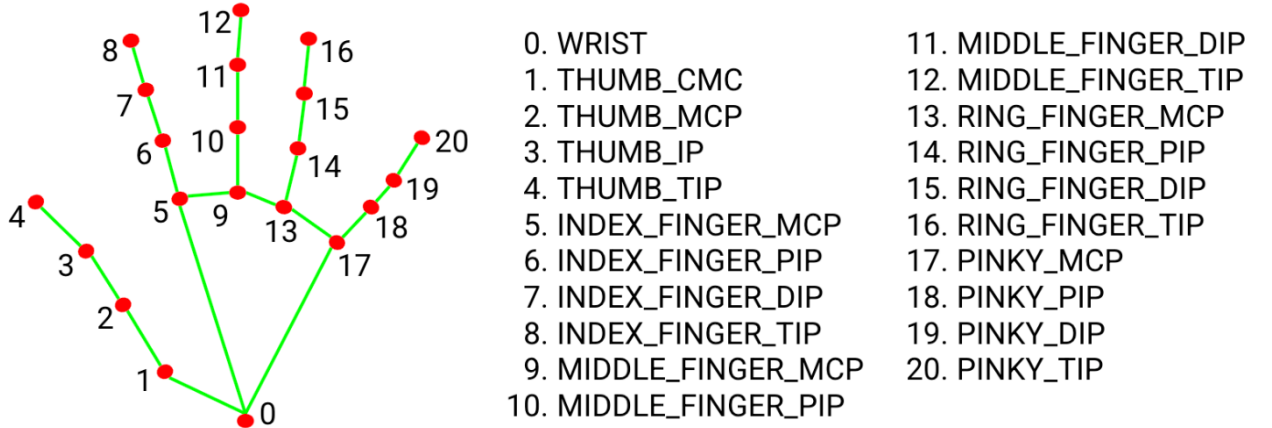


Fig. 8. *MediaPipe* landmarks (Hand landmarks detection guide, 2025)

1.4. Overview of Existing Solutions

In recent years, several projects were developed, researching the possibility of remote hand tracking and signature recognition using a single camera. These solutions aim to create systems that eliminate the need for specialized hardware and physical presence during the signing process. This section will discuss two significant solutions that have made a significant contribution to the development of virtual signature systems.

1.4.1. Finger Air Signing

One notable project for remote signature was proposed by Mascarelli *et al.* (2023). The aim of the project was to create system that allows users to sign documents contactless using only one camera, without additional devices. They managed to implement hand detection in real-time with the high accuracy using *MediaPipe Hands*. The system records the trajectory of the index finger and translates it into a digital signature. For depth estimation, two approaches were used:

- *MiDaS* model that returns the depth map from 2D image.
- Depth based on geometry and triangle similarity.

However, authors abandoned neural networks in favour of geometry, because *MiDaS* was quite slow to work with the live video. Moreover, they mentioned that the operation of the drawing interruption works stably with a good camera and conditions.

1.4.2. Remote Scan Recognition

Another project was developed by Victor *et al.*, (2024). In the work, the focus was on improvement and development of the previous version of the project created by Mascarelli. They managed successfully complete the following task:

- Trajectory smoothing using interpolation algorithms and *Bezier* curves.
- Researching in monocular depth estimation.
- A system for storing signature coordinates and visualizing results was implemented.
- An exploration of different video pre-processing techniques: cropping, masking, and full-image manipulation.

However, the author mentioned that they encountered inaccuracies in determining the hand path, especially with fast hand movement or blurred images, moreover, signature curve smoothing was not robust enough and may result distortions. Also, preprocessing of the video did not improve, but on the contrary worsened the results. These limitations in performance and accuracy were primarily due to hardware constraints, as most of the methods, such as monocular depth estimation, require more computational resources.

2. SOFTWARE DESIGN AND IMPLEMENTATION

This chapter begins with a high-level overview of the system as a whole. From there, it progressively delves into system analysis, including functional, non-functional, user, system requirements and use case diagram. Then, the system design is described using *UML* diagrams such as sequence, component and class. Next, the wireframe and the final user interface development are presented. After that, the projects are built, and an installer is created. Finally, the software is tested.

2.1. System Analysis

This section is dedicated to the analysis of requirements such as functional, non-functional, system, user. It also includes a use case diagram to illustrate the functionality available to each group of users (actors). Additionally, system requirements and user requirements will be considered for the complete analysis of the system.

2.1.1. Functional Requirements

Functional requirements define what the system must do and describe its core features and behaviour. According to Sommerville (Sommerville, 2015), these requirements are influenced by the type of software being developed, the target users, and the organizational standards for writing requirements. Here is a breakdown of the functional requirements:

- The system shall provide users with a list of available cameras.
- The system shall allow users to select the desired camera connected to the *PC*.
- The system shall allow users to start or stop the camera capture at any time.
- The system shall allow switching between *30Hz* and *60Hz*, if supported by the camera.
- The system shall allow users to change the resolution of the video stream.
- The system shall recognize the user's hand in real-time using only one camera.
- The system shall track the position and movement of key points of the hand in space.
- The system shall provide the video screen in real-time captured from the camera.
- The system shall be able to understand and classify gestures of the user.
- The system shall allow users to draw on virtual plane using the gesture *pointing up*.
- The system shall convert the user's hand movement into a digital signature.
- The system shall allow users to clear the canvas manually and by showing *thumbs down*.
- The system shall allow users to save the signature manually and by showing a *thumbs up*.
- The system shall provide users with advanced settings for camera (contrast, saturation etc.).
- The system shall restrict drawing when the user's hand is out of allowed area for drawing.
- The system shall restrict saving the small signature by providing rules for minimum points.
- The system shall require the user to hold gesture for a short time before saving.

- The system shall save signature in *png* format in *signatures* folder.
- The system shall notify the users in the video frame (*FPS*, progress bar, alerts).
- The system shall allow users to configure setting for remote signature (min/max distance, min number of points, saving time).
- The system shall indicate when users are allowed to draw on the virtual canvas.
- The system shall allow the user to enable developer mode to view the predicted hand wireframe by the neural network model.
- The system shall allow users to reset the canvas settings to their default values.

These functional requirements must be met to ensure convenient interaction with the application both manually (via mouse) and remotely using camera and neural networks. Moreover, by following these requirements, strict boundaries are set for what is in the scope of project, which helps by saving time and accelerating the development of the application.

2.1.2. Non-functional Requirements

Unlike functional requirements, non-functional requirements describe not what the system does, but how it does it. As Sommerville (2015) notes, such requirements concern overall performance, reliability, usability, and other characteristics that are important for the perception of the system as a whole. They often determine whether the system will actually be usable. Here is a breakdown of the non-functional requirements:

- *Performance*. The system must provide a minimum refresh rate of at least ten frames per second. On more powerful hardware, the system is expected to consistently deliver a frame rate of at least *30 FPS*.
- *Ease of use*. The user should be able to master the basic functions of the system within 5 minutes; the interface contains no more than three screens with tips. The interface provides intuitive icons to support user experience.
- *Size*. The program should take no more than one gigabyte of disk space.
- *Reliability*. The system must be able to operate continuously without failure and allow users to save an unlimited number of signatures (limited only by disk space). The system must maintain video capture of at least 10 minutes.
- *Portability*. The system should be able to run on any platform that supports *Python* version higher 3.9, including *Windows 10* and *11*, *Linux*, and *macOS*.

These non-functional requirements are critical to ensure that the system meets the expectations of users in terms of performance, usability, reliability, and portability. This guarantees that the system not only performs its tasks but also provides stable user experience across different platforms and hardware equipment.

2.1.3. System Requirements

This subsection is devoted to system requirements that includes both the hardware and software conditions necessary for proper functioning of the application. Here will be minimum and recommended requirements for hardware and used software in the development.

The table below (see table 1) demonstrates what are the minimum requirements and recommended requirements. The recommended requirements were create based on the tested hardware, moreover, it provides the alternative hardware for *AMD* users. The tested setup was able to work both on *30 FPS* and *60 FPS*. The minimum requirements are supposed to work on *30 FPS*, which is enough for this system.

table 1

Hardware Requirements

Component	Minimum Requirements	Recommended (Tested Setup)
<i>CPU</i>	Intel Core i5 (6th Gen) / AMD Ryzen 5	Intel Core i7-6700 / AMD Ryzen 7 1700
<i>RAM</i>	8 GB DDR4	16 GB DDR4
<i>GPU</i>	Integrated GPU with OpenGL 3.3 support	NVIDIA GTX 1070 / AMD Radeon RX 580
<i>Camera</i>	HD 720p 30 FPS	Full HD 1080p 60 FPS
<i>Disk Space</i>	500 MB free space	500 MB free space
<i>Operating System</i>	Windows 10, 11 / macOS / Linux	Windows 10, 11 / macOS / Linux
<i>Others</i>	USB port	USB 2.0 port

The table (see table 2) represents a list of software components, which are required for system implementation. It involves the version and purpose of each component which will help ensure the correct operation of the program. The most important components are *Python*, *OpenCV* and *MediaPipe*, which perform core functions in development, such as video processing, hand tracking and gesture recognition. Moreover, the table lists libraries and tools for creating *GUI* (Graphical User Interface), such as *PySide6* and its additional modules.

table 2

Software Requirements

Component	Version	Purpose
<i>Python</i>	3.10 or higher	Core programming language
<i>OpenCV</i>	4.1 or higher	Video capture and image processing
<i>MediaPipe</i>	0.10.22 or higher	Hand tracking and landmark detection
<i>NumPy</i>	Any (used 1.26.4)	Numerical operations and data structures
<i>PySide6</i>	6.9.0	Graphical User Interface
<i>PySide6 Addons</i>	6.9.0	Additional GUI modules and widgets
<i>PySide6 Essentials</i>	6.9.0	Core components for GUI development
<i>Camera Driver</i>	Latest (compatible)	Drivers for camera interaction

2.1.4. User Requirements

User requirements describe what tasks a user should be able to perform using a system and how the user expects to interact with it. These requirements are formulated from the user's perspective and guide the development towards convenience, clarity, and achievement of the user's goals. According to Sommerville (2015), user requirements should be written in clear language to both developers and non-technical stakeholders. Below is a breakdown of the user requirements:

- Users must be able to quickly choose a camera and start capturing without configuration.
- Users must be able to interact with the system with the help of gestures of the hand and without additional equipment (except camera).
- Users must be able to draw on the virtual canvas and see the result in real-time.
- Users must be able to save the signature with the help of intuitive gestures.
- Users must be able to clear the canvas with the help of intuitive gestures.
- Users must be able to get clear feedback from the system while interacting with it.
- Users must have the option to manually clear or save the canvas via buttons.
- Users must be able to configure camera and signature (recognition) settings.
- Users must be able to interact with the system without training or technical knowledge.

By following these user requirements, the development shifts towards how it would be best for the user to interact with the system, ensuring that the system becomes easy to use. In this way, a conceptual model of how the user will perceive and engage with the final application is created.

2.1.5. Use Case Diagram

At the highest level, the system involves three primary actors that define the interaction between the user and the virtual signature environment. Each actor plays a role to support the signature recognition process:

- *User* – the primary human actor who interacts with the system.
- *Camera* – external hardware that provides system with video input.
- *File System* – system that stores saved signatures.

The system also describes the use cases that are performed by the described actors. The system includes the following cases:

- *Start/Stop Capture* – allows the user to initiate or terminate the video capture process.
- *Configure Camera Settings* – enables the user to choose a specific camera, fine-tune frame rate, resolution, and other advanced parameters.
- *Draw Signature* – lets the user sign by performing hand gestures, which are tracked and interpreted by the system.

- *Save Signature* – gives the user the ability to save the completed signature to the folder *signatures* by showing *thumb up* or clicking the button *save*.
- *Clear Signature* – permits the user to erase the current drawing and begin a new signature by showing *thumb down* or clicking the button *erase*.
- *Adjust Recognition Settings* – allows the user to change system parameters such as minimum and maximum gesture distance, minimum number of signature points, and required hold duration for saving.

The use case diagram below (see Fig. 9) shows how the main actors interact with the system. Most of the actions are performed by the user, such as capturing video, drawing, and saving, or adjusting settings. The camera provides video input required for gesture recognition, hand tracking, while the file system manages the storage of saved signatures.

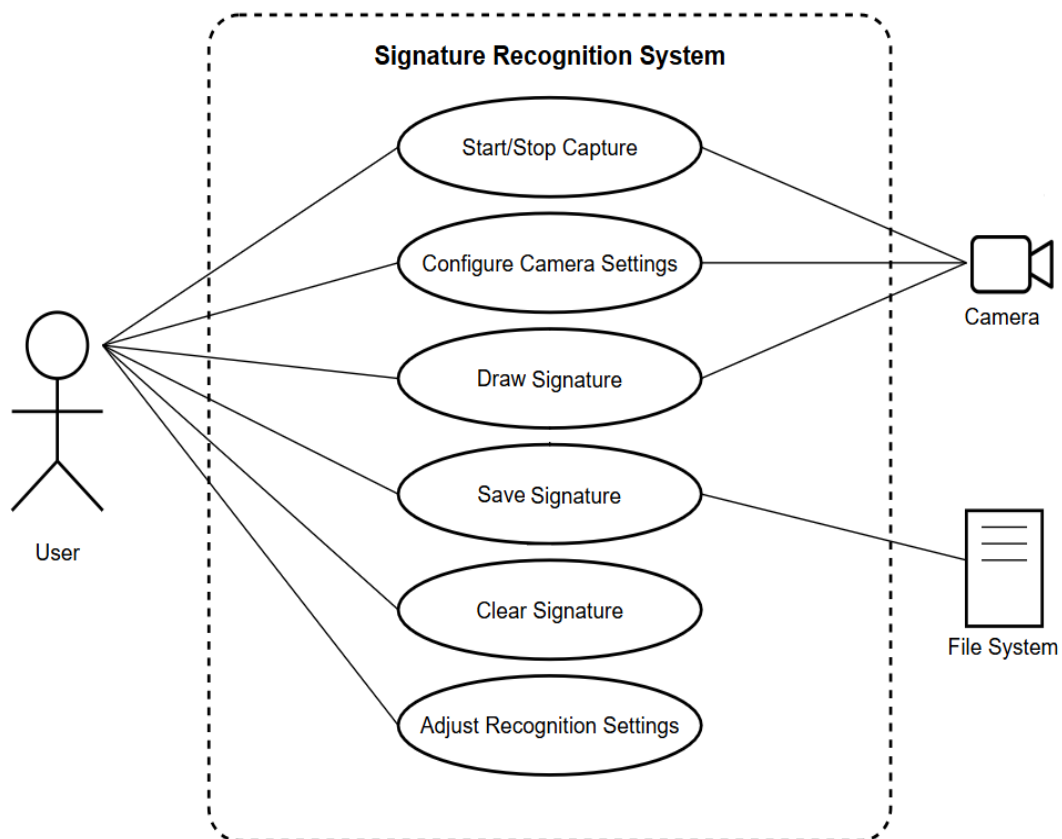


Fig. 9. Final use case diagram

2.2. System Design

This section focuses on the system architecture. It includes a class diagram to illustrate the relationships between classes, along with their attributes and methods. A component diagram is also provided to depict the system's modular structure and the interactions between its components. Additionally, a sequence diagram is presented to demonstrate the dynamic behaviour of the system by showing how objects interact over time to perform a specific use case.

2.2.1. Sequence Diagram

The sequence diagram (see Fig. 10) below describes the process of capturing real-time video in *UI* (User Interface). User initiates the process by clicking the button *Start* in *CameraSettingsDock*. It triggers the method *onToggleCapture()*, which in turn starts the *VideoThread* thread via the *start_th()* method. Inside the thread, an active flag is set, and the main loop is started. The thread then initializes access to the *Camera Device* using *cv2.VideoCapture()*. After a successful connection, the thread runs a loop in which it requests frames from the camera using *cap.read()*. Then it processes the obtained images and passes them back via the *ImageUpdate.emit(image)* signal, updating the video window. After starting the capture, the interface changes the button icon to *Stop*.

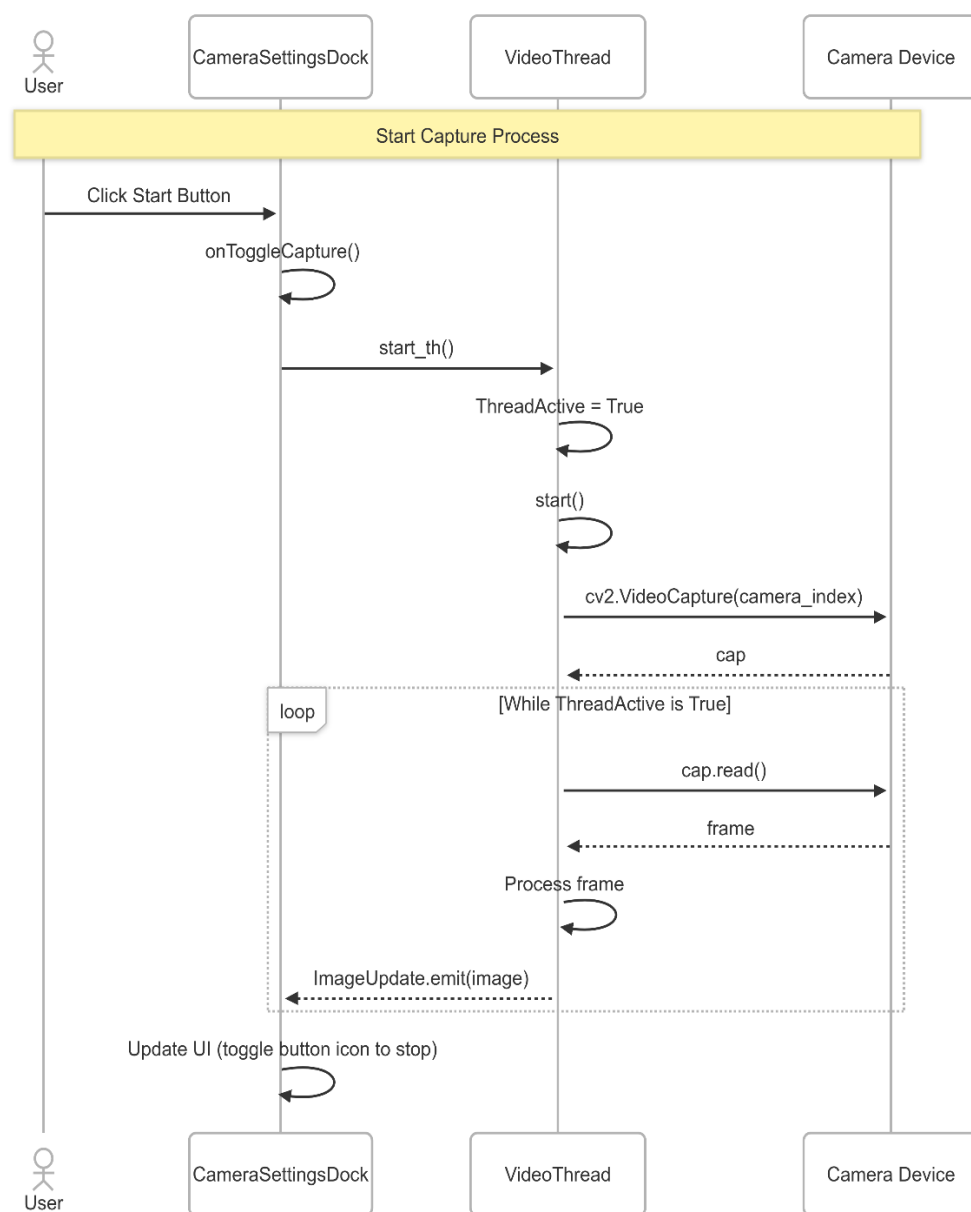


Fig. 10. Final Sequence Diagram “Start Capture”

In the following sequence diagram (see Fig. 11) is demonstrated the process of changing camera settings by the user. Firstly, user selects the desired camera from the dropdown list, that triggers event *onCameraChanged* of *CameraSettingsDock*. Then it calls the method *change_settings()* of class *VideoThread* to update the settings. After changing the settings, the thread is re-started/started using *start_th()*. The frame rate and resolution settings work the same as camera selection by triggering *onFPSChanged* or *onResolutionChanged* events.

Additionally, it is possible to open advanced settings when the button is pressed, the *onClickAdvancedSettings* event is triggered, after which it calls the method *run_camera_settings()* to open the settings dialog implemented by *OpenCV* library. Changes in advanced settings do not stop the thread (video capture), thus settings such as brightness, contrast, hue, sharpness, gamma etc. can be modified dynamically.

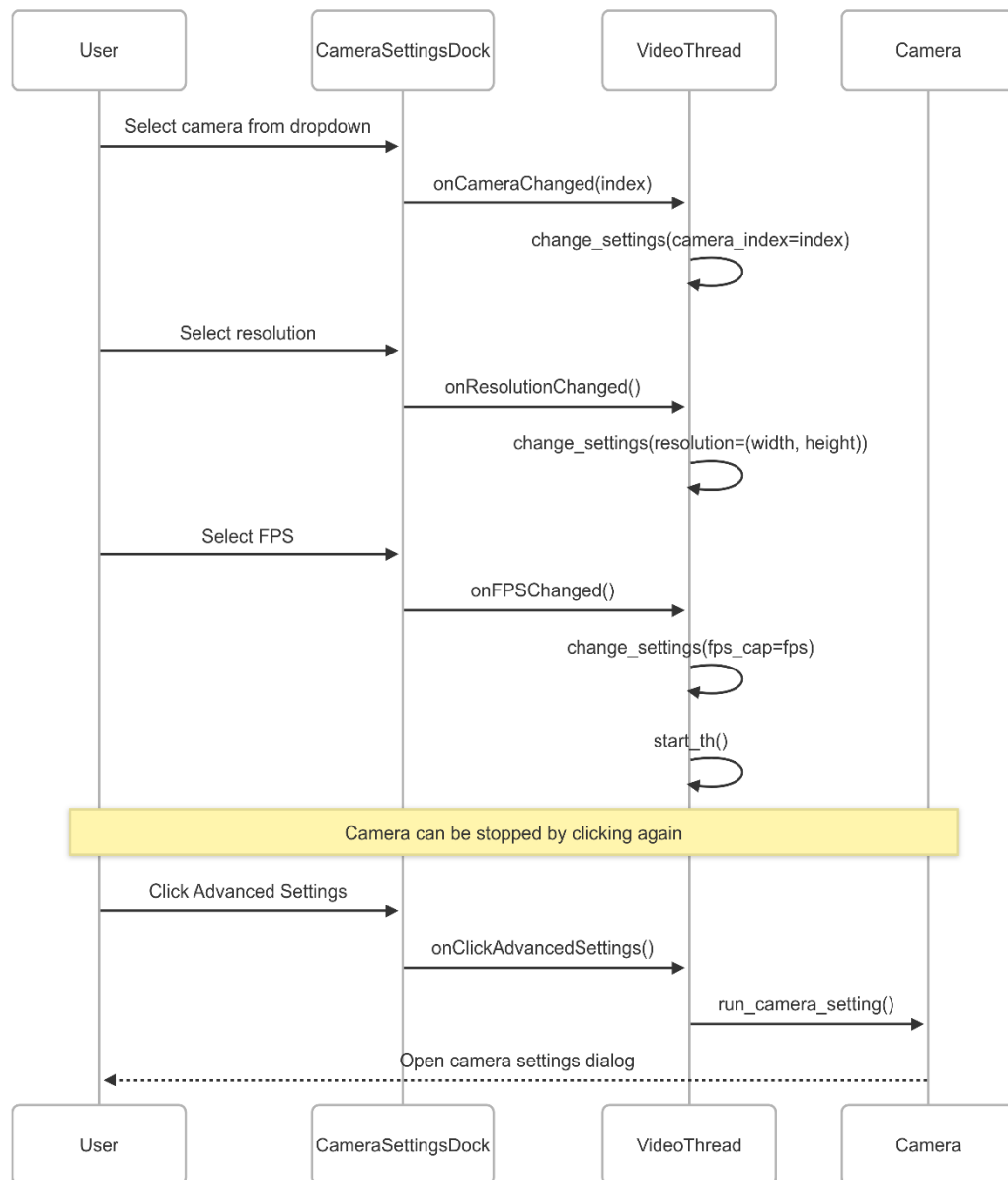


Fig. 11. Final sequence diagram for “Configure Camera Settings”

The diagram (see Fig. 12) user interaction with the system starts from working with a camera that reads a video stream using the *cap.read()* method, passing it to a separate video processing thread. This thread passes frames to the gesture recognition model (*SignatureRecognition*), then callback function *get_result()* is called to return the gesture result. Inside the *handle_gestures()* function, gesture-based logic is executed: if gesture is “*Pointing_Up*” and the distance is valid, the drawing mode is activated, (*x, y*) positions are calculated, and if previous positions exist, a line is drawn. If the gesture is “*Thumb_Up*” and the signature is valid, the signature is saved, and the drawing board is cleared. If the gesture is “*Thumb_Down*”, the drawing board is simply cleared. Finally, the frame is updated with the drawing board overlay for visualisation.

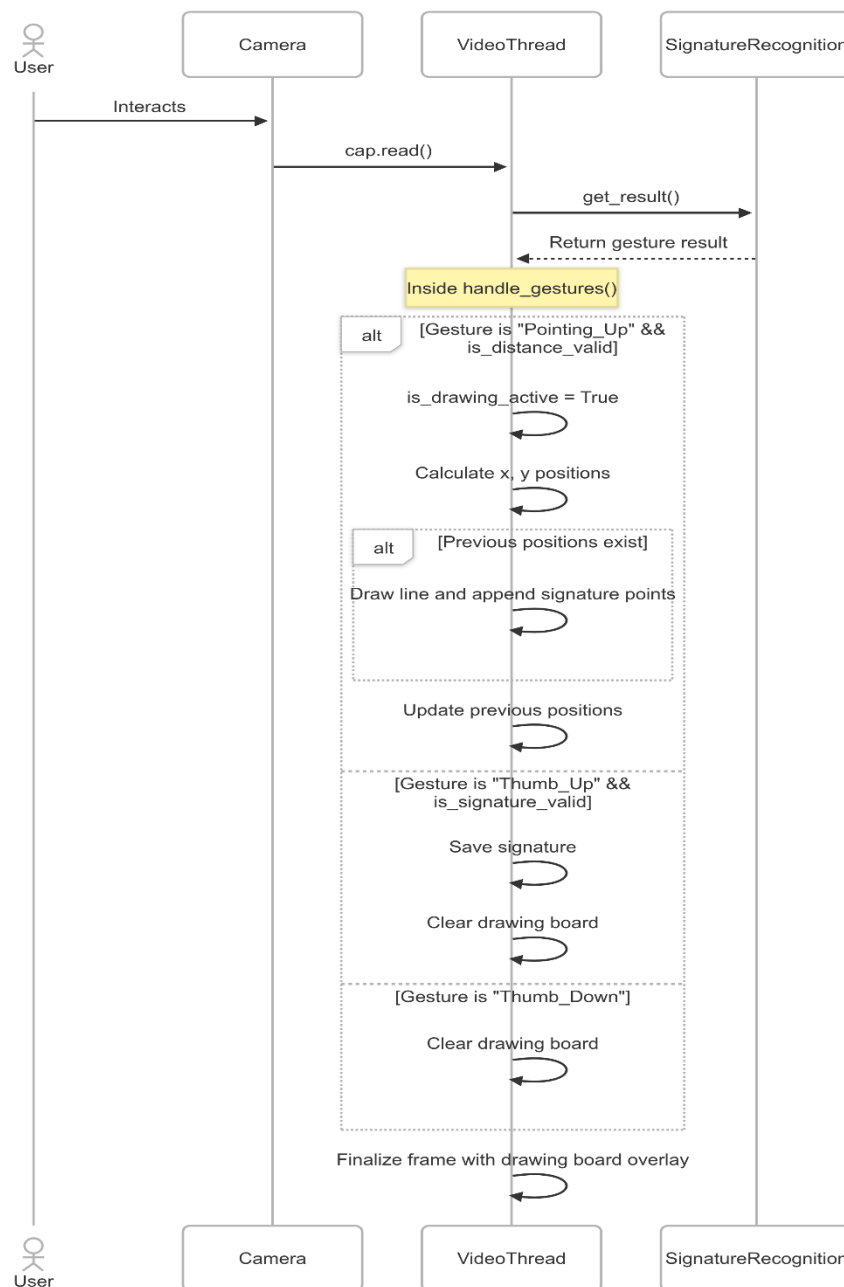


Fig. 12. Final sequence diagram for “Gesture/Signature Recognition”

This sequence diagram (see Fig. 13) describes the process of adjusting signature settings. It starts with the user interface (*SignatureSettingsDock*), changing different parameters. When user changes the minimum distance for gestures, the interface updates the corresponding setting in the video processing thread (*VideoThread*). The new value for *min_distance* parameter is set using sliders. The same happens with the other parameter. Also, the user can turn developer mode on or off, which changes the *dev_mode* parameter to see wireframe of detected hand. Moreover, the user can reset all settings by pressing the reset button. All changes take effect when the next frame is processed. The system also provides buttons for manual saving and clearing canvas (it is not shown in this diagram, because it works in the same way as discussed in previous sequence diagram).

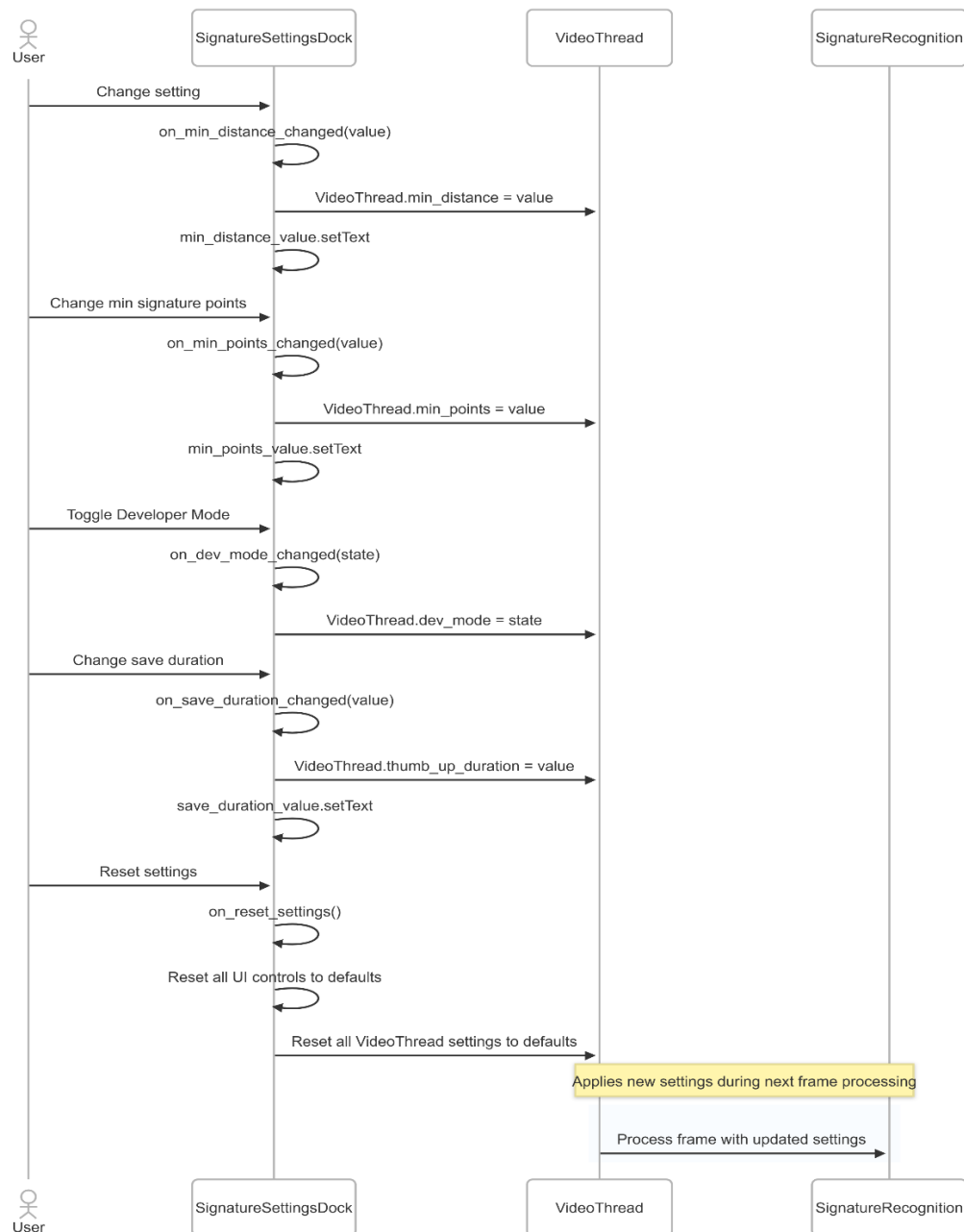


Fig. 13. Final sequence diagram for “Adjust Signature Settings”

2.2.2. Class Diagram

The class diagram (see Fig. 14) describes the architecture of the application for remote signing and gesture recognition with only one camera. The *VideoThread* class is responsible for the video stream processing thread, including image capture, gesture recognition, signature board management, and result validation. It uses *SignatureRecognition* to interact with the gesture recognition model. The central element of the user interface is the *MainWindow*, which contains instances of the *VideoThread*, *CameraSettingsDock*, and *SignatureSettingsDock* classes, which provide an interface for configuring camera and signature settings, respectively. Both dock-widgets control the *VideoThread* parameters, such as resolution, FPS, distance sensitivity and the number of signature points. Thus, the system is built modularly, with the separation of processing logic, user interface and parameters, ensuring scalability and ease of management.

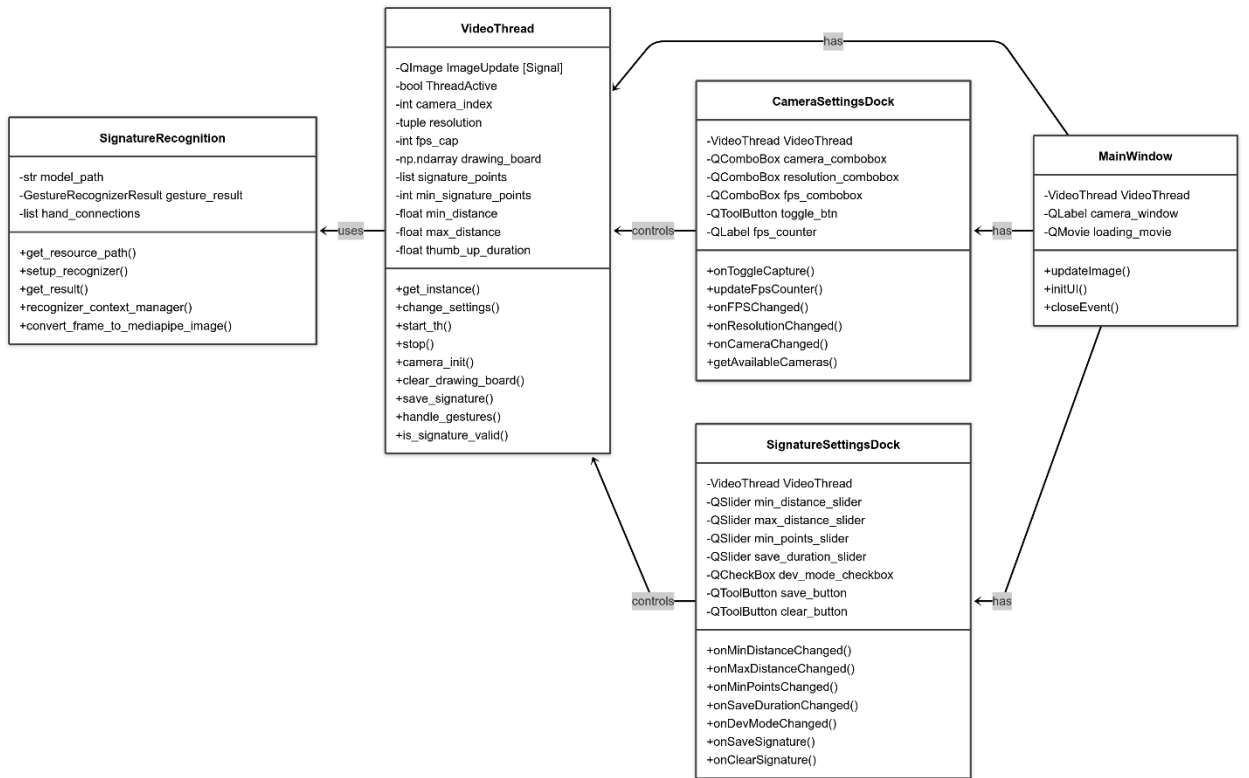


Fig. 14. Final class diagram

2.2.3. Component Diagram

The component diagram (see Fig. 15) shows the high-level architecture of remote signature system, which is divided into three subsystems each involves key components with functionalities:

- *Application*. It includes *Video Window* which responsible for displaying user's actions (signature, hand wireframe, notification) and video. Also, it includes *Camera Setting Panel* and *Signature Setting Panel* allowing to manage settings of the camera and signature.

- *Video Processing*. This subsystem is intermediate between *Application* and *Signature Detection* subsystems. It includes *Camera* to obtain frames and then pass it to *Signature Detection*. *Canvas* component displays overlay information and can be cleared, modified, and saved. *Settings* component interacts with *Camera Setting Panel* and *Signature Setting Panel* to apply current settings (in the case of camera settings it restarts the video stream).
- *Signature Detection*. It involves *Preprocess* component to prepare/convert frames in required format for neural network. Then, it interacts with *Hand Detection*, *Gesture Recognition* and *Depth Estimation*. These components are encapsulated and performed as one action in the current project. Finally, it returns the obtained information to the *Video Processing* subsystem.

This component diagram demonstrates a modular system architecture, where each subsystem block has defined methods and interactions. This approach describes, at a high-level, the responsibilities, providing a complete picture of what is happening within the system.

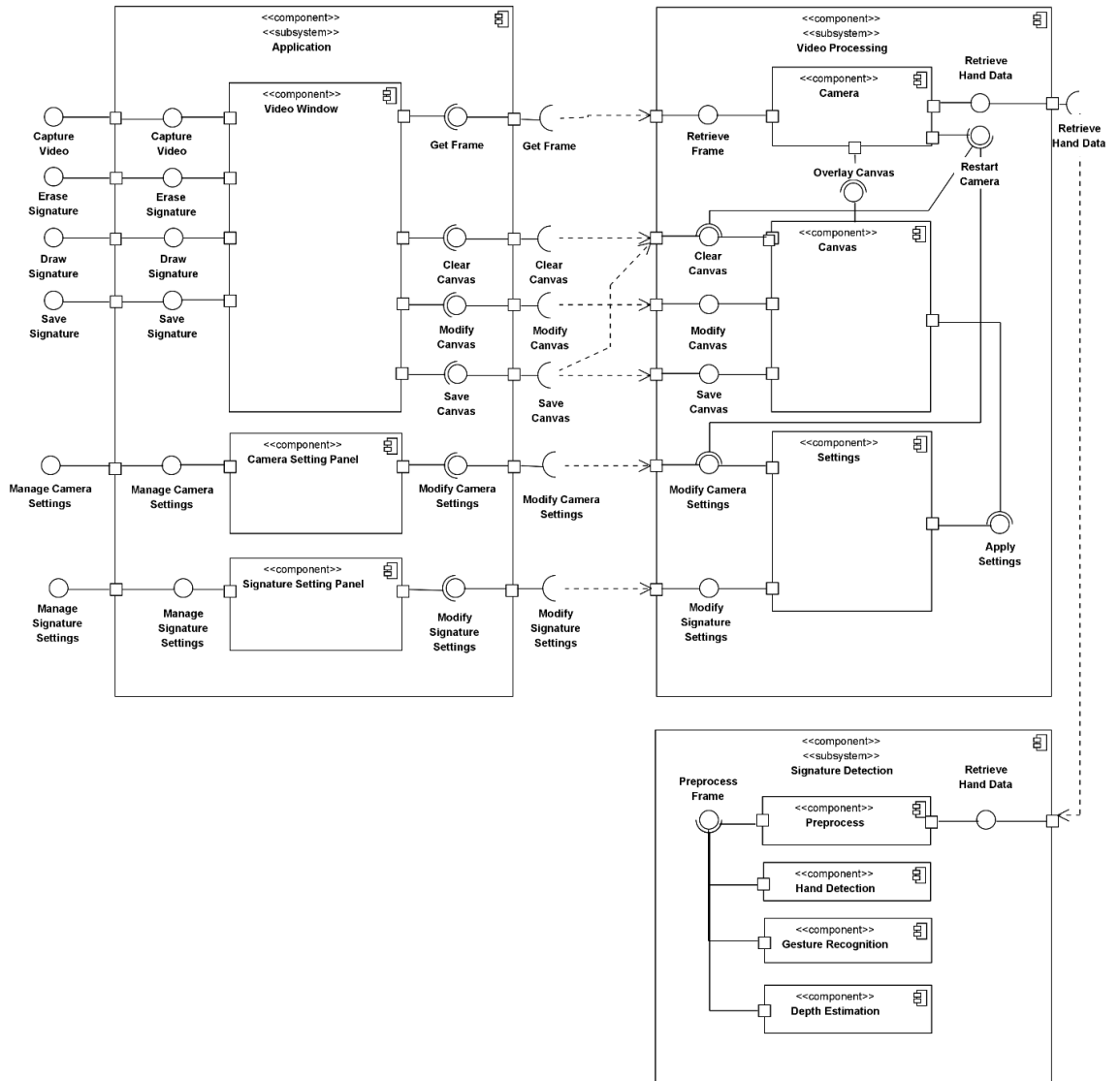


Fig. 15. Final component diagram

2.3. Software Implementation

This section describes the implementation of the system, including wireframing, development of the graphical user interface. The implementation was done using *Python* and the *PySide 6* framework for *GUI*, with external libraries such as *OpenCV* and *MediaPipe*. Furthermore, building and the installer creation for software will be covered.

2.3.1. Wireframe

The UI/UX play a key role in creation of the product, providing convenient interaction and user's satisfaction. Well-designed interfaces increase the efficiency of use of the system but also contributes to the achievement of the business goal. As Garrett (2010) noted, UX is about how users perceive and interact with a product, rather than its internal workings. The wireframe provides a simple representation of the application. As shown in the image (see Fig. 16) it serves as the initial step in the UI design process, outlining three main components:

- *Left Dock Panel*. For configuring camera settings, starting/stopping video capture, and accessing external settings provided by the camera driver.
- *Middle Window*. Displays the real-time video stream and system feedback.
- *Right Dock Panel*. Used to adjust signature parameters via sliders, enable developer mode, and manage signature actions (save, clear, reset).

Having developed the wireframe, it becomes possible to proceed with the implementation of the *GUI*. The structure provides a clear understanding of the arrangement and expected behaviour of each *UI* component, which simplifies the transition to the development phase.

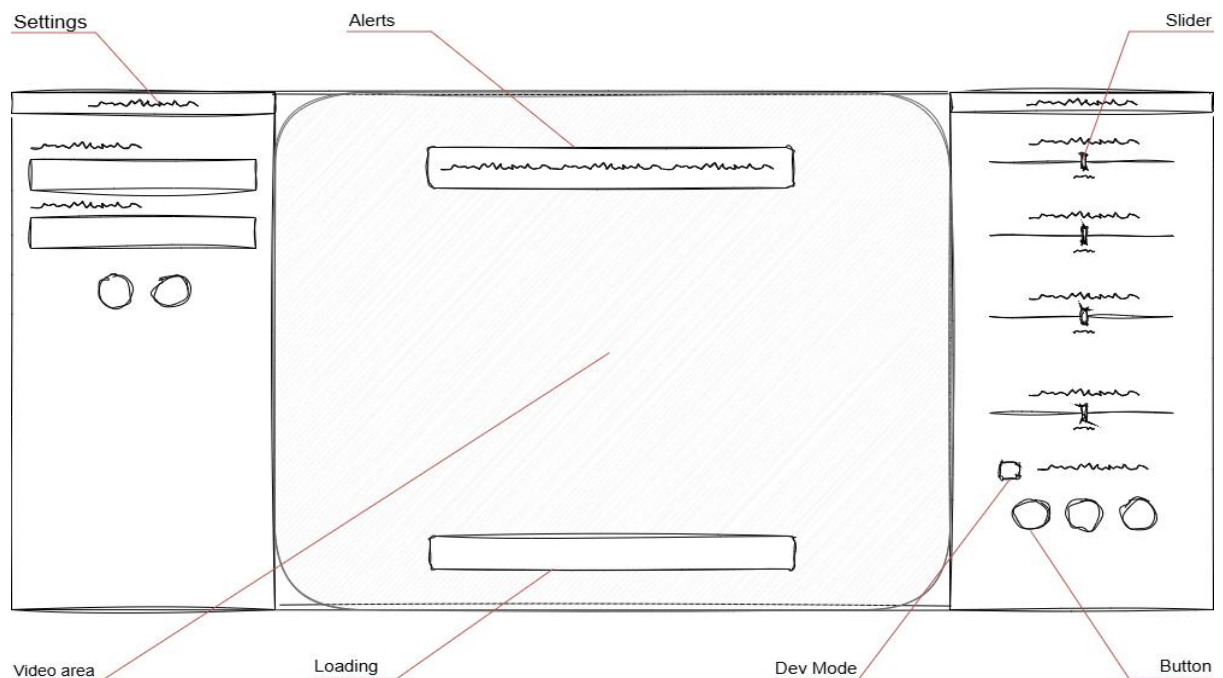


Fig. 16. Wireframe of the system

2.3.2. Graphical User Interface

Following the wireframe structure, the graphical user interface (see Fig. 17) was successfully developed using *PySide 6*. The *GUI* incorporates several design improvements that enhance the overall user experience and functionality of the application. These include:

- The side panels *Camera Settings* and *Signature Settings* are now visually distinct, with a light blue background and darker headers. The *Central Window* has increased contrast to draw user attention during video capture.
- A *Menubar* appeared at the top, consisting of *About* and *Help*, to show information about the application and also to help the user master the application quicker.
- There is now a status bar at the bottom that shows the number of points on the canvas in blue text and the location of the index finger in yellow along the x and y coordinates.
- The buttons now contain intuitive icons that reveal tooltips when hover over them, making it easy to quickly learn how to interact with the app.
- In the *Signature Settings* panel, sliders are styled in blue, each with a label and current value displayed beneath them.
- In the *Camera Settings* panel, dropdown lists are styled more clearly with labels. At the bottom, the FPS counter blends with the sidebar background when the video is inactive. When active, it turns green, yellow, or red depending on the FPS (good, medium, or low).

As a result, the modern and appealing graphical user interface was designed. This makes the application both functional and user-friendly.

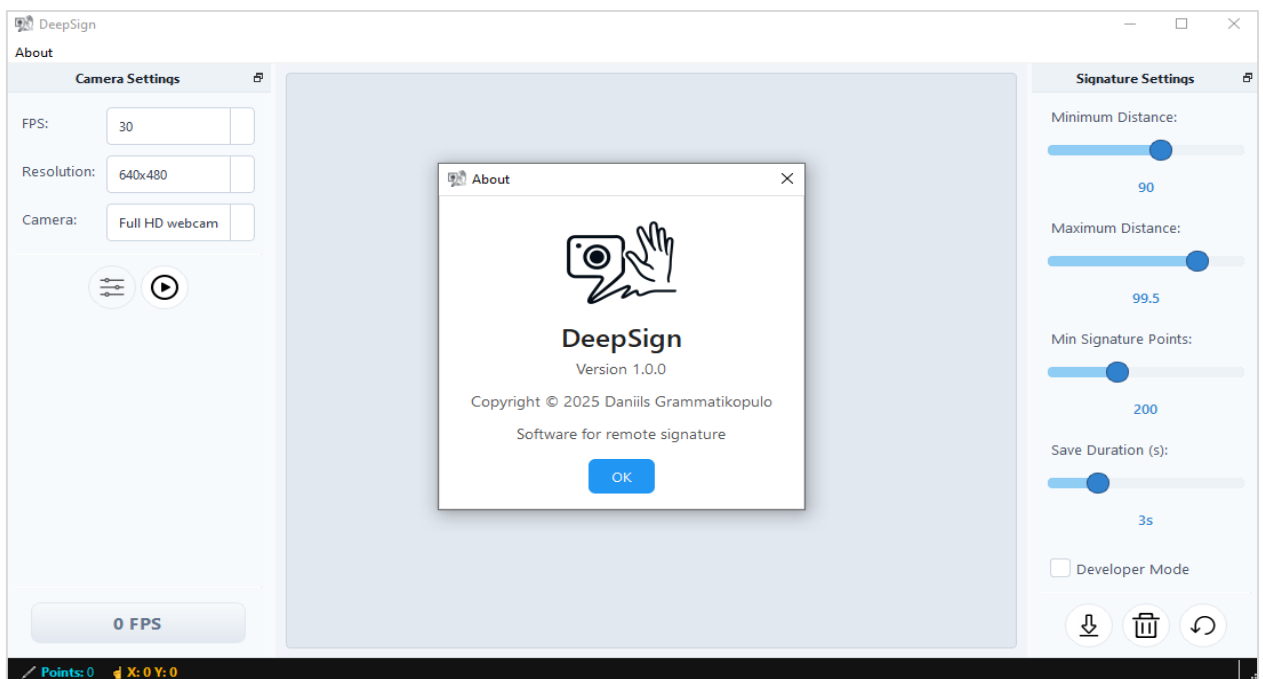


Fig. 17. Final UI of the application

2.3.3. Software Packaging and Installer Creation

As the first step, the project is built using *PyInstaller*. It allows to create the executable file and the *_internal* folder, which consists of the source code and libraries, by writing the *main.spec* file that includes the configuration and instructions on how the project should be built. In particular:

- Includes required data and resources (e.g., models and icons).
- Lists hidden imports such as *cv2*, *mediapipe*, and *PySide6* components.
- Generates an executable named *DeepSignus* with a specified icon.

The next step involves creating an installer using *Inno Setup*. This is achieved by writing a script file named *installer.iss*, which contains configuration for generating the final executable file that runs setup procedure. Configuration file consists of the following instructions:

- The application name (*DeepSignus*) and version (1.0).
- Installation path, path to license and other setup parameters.
- Files to be included: the main executable and the internal resource folder.
- Shortcuts to be created on the desktop and in the Start Menu.
- The *DeepSignus.exe* application is launched automatically after installation.

As a result, the installer was created (see Fig. 18). First, it displays the license. Then, it shows the selection of the desired installation folder (the application location). Next, it prompts the user to select the *Start Menu folder* to set up the shortcut. After that, it asks whether to create a desktop shortcut. Then, the installation process begins. At the end, there is a checkbox labelled “*Launch DeepSignus*” that allows the application to start immediately after closing the setup wizard.

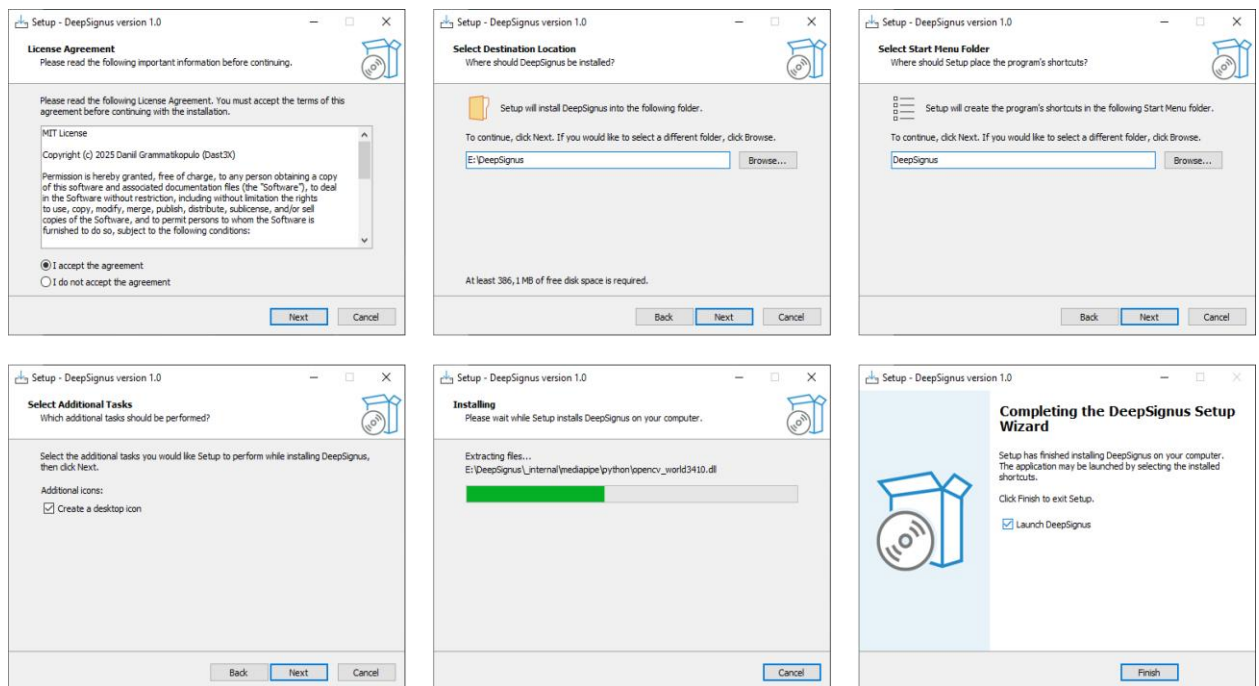


Fig. 18. Setup wizard for application

2.4. Software Testing

This section is about testing of the application. The software was tested using both manual and automated approaches. Manual testing was performed to evaluate the usability and behaviour of the application. This approach was useful for assessing the graphical user interface and user interactions, such as adjusting settings, capturing video etc. Unit testing, on the other hand, was used to verify the correctness of components and algorithms. Together, these two testing methods provided a balanced coverage for technical correctness and user experience.

2.4.1. Manual Testing

The image below (see Fig. 19) shows that the application launched successfully, the start and stop capturing worked as expected. Furthermore, the settings worked: *FPS* can be switched from 30 to 60; resolutions can be chosen among 640x480, 1280x720 and 1920x1080; cameras can be switched. Each of the described features restarts the video stream, providing user with the loader. Similarly, it was managed to configure *Signature Settings*. When the “Developer Mode” flag is enabled, the system visualizes the hand skeleton using red dots at key joints and white lines connecting them, confirming that this feature is working correctly. Additionally, the video stream window displays the warning “Too close to camera! Please move back.”, which indicates that the logic for checking the distance from the camera has been triggered.

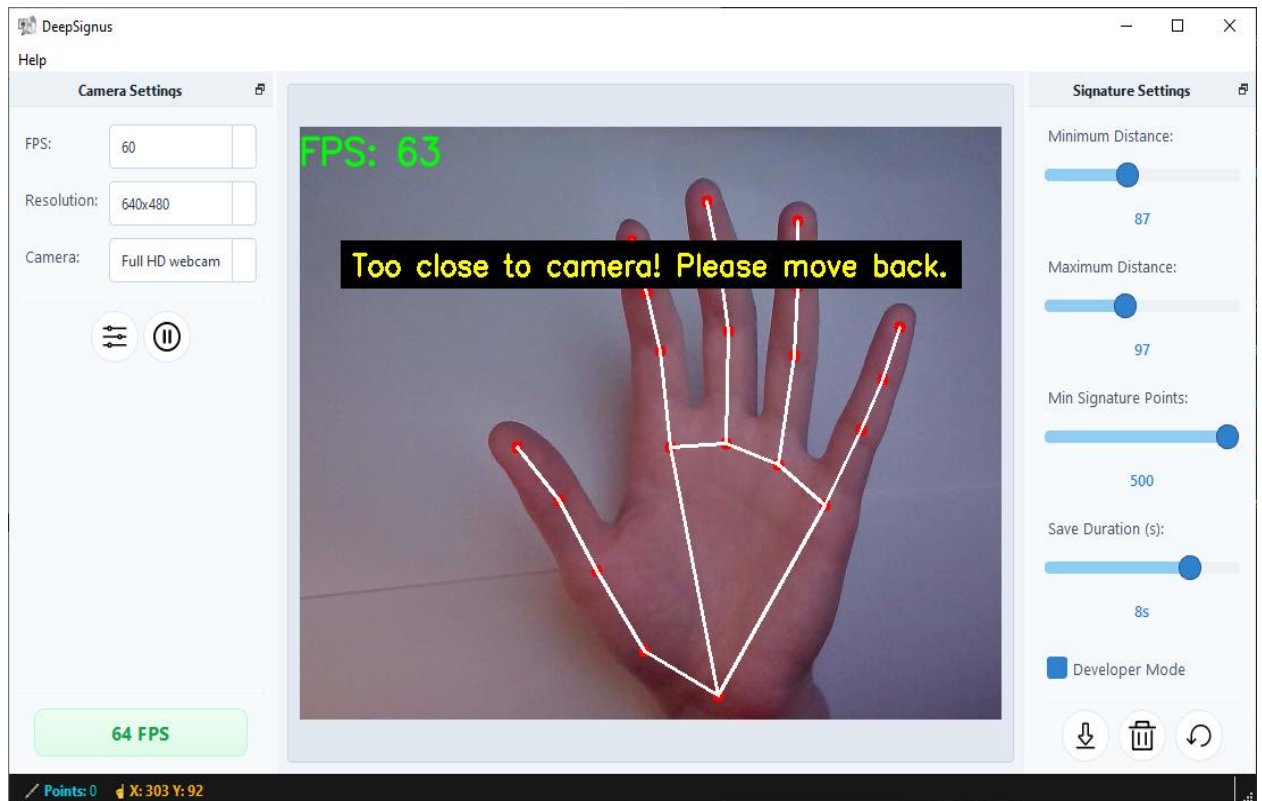


Fig. 19. Manual testing of signature settings

On the picture below (see Fig. 20) the signing procedure being tested using the gesture of an upward-pointing index finger. During the test, the symbols “T”, “S” and “I” were drawn in the air with the index finger, which were displayed on the screen as a green line. The program also showed that the interrupts worked well, which made it possible to write the letters separately from each other. The system tracked the movement and collected coordinates, reaching the set threshold of 500 points, which is confirmed by the progress bar at the bottom of the screen (100% / 500 points).

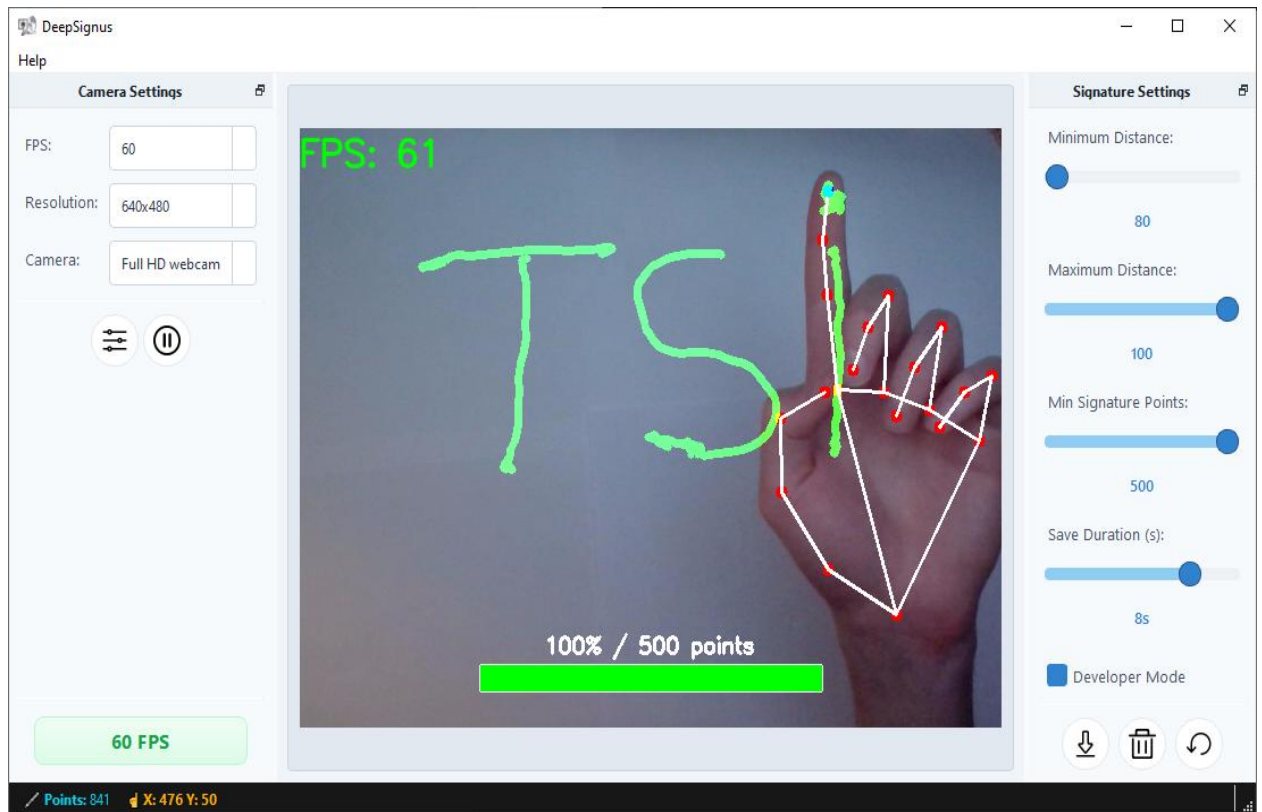


Fig. 20 Manual testing of signing

The next picture (see Fig. 21) illustrates the testing of the signature saving process, where the output is stored in *PNG* format in the */signatures* folder. In this test, the saving process was triggered using the “*Thumbs Up*” gesture, as evidenced by the yellow progress bar displaying the text “*Saving 54%*”. Given that the number of points is 272, which exceeds the threshold of 200, the saving process was permitted. However, if the threshold is not exceeded, then the system will notify at the bottom of the video stream with the message “*Not enough points! Need <remaining amount> more*”. It ensures that signature is saved when a sufficient number of points is collected. Additionally, the signature can be saved instantly by pressing the *Save* button at the bottom of the right panel. The clearing procedure works in a similar way but is triggered by the “*Thumbs Down*” gesture. Alternatively, the signature can be cleared manually by clicking the *Clear* button, allowing users to reset the signature input without performing the gesture.

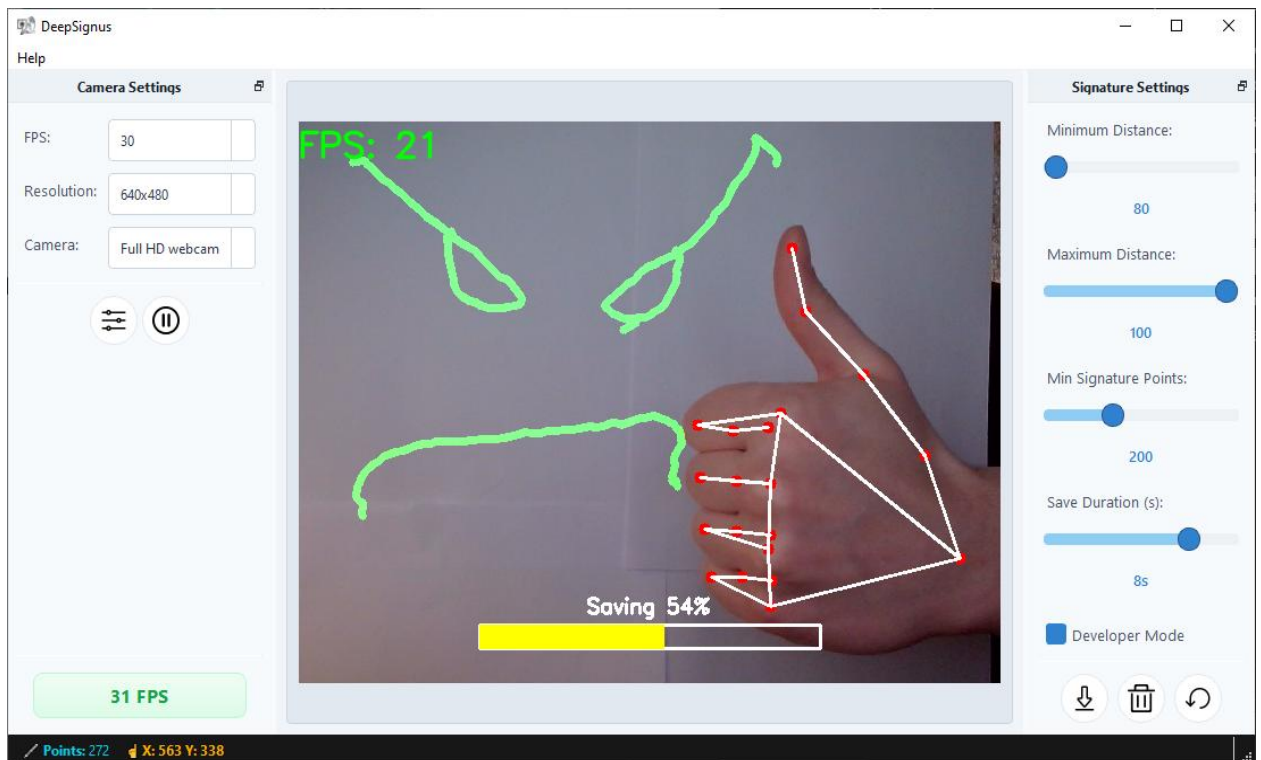


Fig. 21. Manual testing of signature saving

The next image (see Fig. 22) demonstrates the advanced camera settings accessible from the left settings panel. These settings, integrated via the *OpenCV* library, provide low-level control over the camera's hardware properties and are divided into two main tabs:

- *Video Proc Amp*. It allows to change settings such as brightness, contrast, hue, saturation, sharpness, gamma, white balance, backlight comp and gain.
- *Camera Control*. It allows to change settings such as zoom, focus, exposure, pan, tilt, roll.

Fine-tuning these parameters can enhance hand recognition and tracking accuracy by making the hand stand out more clearly from the background.

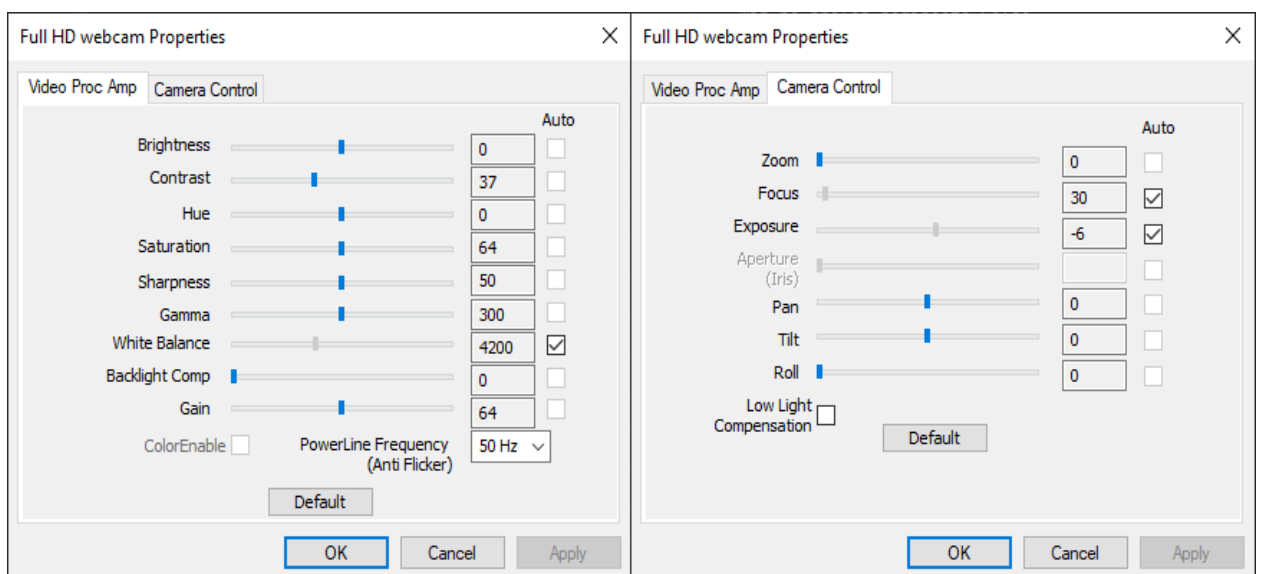


Fig. 22. Advanced Settings

The application also includes a User Guide, which can be accessed from the Help menu in the menu bar or by pressing F1. This ensures that users can navigate the application independently. The User Guide includes the following tabs:

- *Overview*. Brief overview and purpose of the application. Here the user learns that the system allows creating virtual signatures using a single camera.
- *Camera settings*. Tab for explanation of camera settings. Here the user can read about the settings and get the recommendations for the correct operation of the system.
- *Signature Settings*. Tab for explanation of the signature settings. Here the user can read about the settings and get recommendations how to draw and position the camera.
- *Usage Guide*. This tab describes a step-by-step guide to using the virtual signature system. It explains how to get started with the application: launch it, connect a camera, and so on.
- *Gesture Guide*. This tab explains how to use gestures to control the app, including drawing, clearing, and saving a signature. The gestures include raising the index finger to start drawing, the “thumbs down” gesture to clear the canvas and the “thumbs up” gesture to save the signature. It also provides tips for improving gesture recognition.
- *Troubleshooting*. This tab helps resolve common issues with the application. It addresses problems such as the camera not working, gesture recognition issues, drawing problems.

The picture below (see Fig. 23) demonstrates the *Gesture Guide* for the user. As can be seen, it not only includes a description of the gesture but also images to enhance understanding.

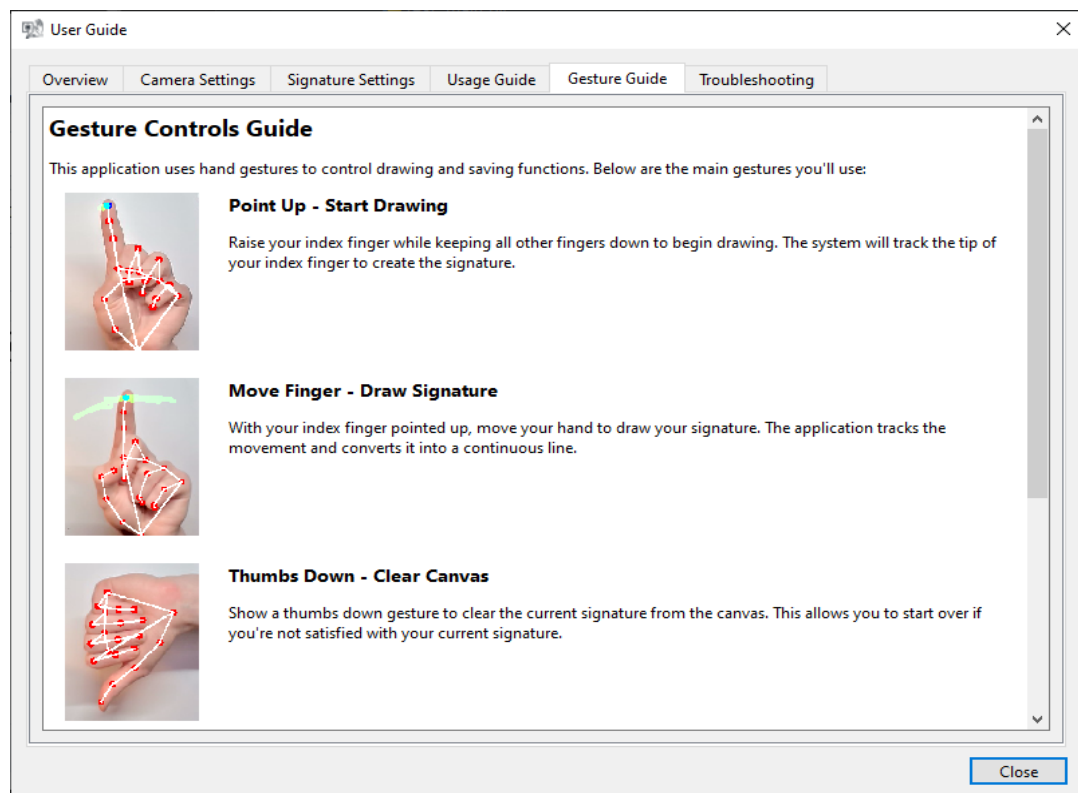


Fig. 23. Gesture guide

2.4.2. Unit Testing

Unit testing was conducted using the *unittest* library, which is part of *Python*'s standard library, along with the *QTest* module from the *PySide 6* framework for testing user interface. The tests were organized in a dedicated */test* directory. This directory includes a */scripts* folder, which contains unit tests written for each component of the application.

All test scripts follow the naming convention *test_*.py* which is automatically discovered by script *run_tests.py* (located in */test* directory). This script runs all the tests and then gathers information about the results. At the end, it creates the table which consists of:

- *Test File*. The name of the script, starting with following convention *test_*.py*.
- *Total Tests*. The total number of tests in the script.
- *Passed/Failed/Errors*. The total number successfully passed/failed/error-ended unit tests.
- *Time*. Execution time of the script.
- *Status*. The status of the script: *PASSED* if there were no fails or error, otherwise *FAILED*.

The last row of the table provides the overall summary, aggregating the total number of passed, failed, and error-ended tests across all scripts, along with the total execution time. Then, it calculates the pass rate based on all tests and saves the table in markdown file *test_result.md*. As shown on the picture below (see Fig. 24), providing a breakdown of the unit test results for each script. The unit testing consisted of 9 scripts with a total of 64 test cases. All tests passed without any failures, and the total execution time was just 2 seconds.

Run Date: 2025-05-11 20:15:38						
Test File	Total Tests	Passed	Failed	Errors	Time (s)	Status
test_about.py	3	3	0	0	0.382	PASS
test_camera_settings_dock.py	9	9	0	0	0.664	PASS
test_help_dialog.py	10	10	0	0	0.063	PASS
test_main.py	7	7	0	0	0.668	PASS
test_signature.py	12	12	0	0	0.008	PASS
test_signature_settings_dock.py	11	11	0	0	0.072	PASS
test_statusbar.py	2	2	0	0	0.223	PASS
test_utils.py	2	2	0	0	0.001	PASS
test_video_thread.py	8	8	0	0	0.018	PASS
TOTAL	64	64	0	0	2.099	PASSED
Pass Rate: 100.00%						

Fig. 24. Screenshot of unit tests result

3. EXPERIMENTAL ANALYSIS AND FUTURE IMPROVEMENTS

This part is devoted not only to directions for improving the developed system, but also to the description of the conducted experiments and their parameters. Based on the results of implemented system, presented in the previous part. The aspects that can affect the accuracy and stability of the system are presented here. In particular, the influence of the number of frames fed to the neural network input, potential improvements such as background removal using depth estimation models that may be implemented in future versions.

3.1. Frame Count Optimization for Neural Network Input

Increasing the number of input frames to the neural network can be seen as a double-edged sword: while it may improve the accuracy (track hand at high speed; signature is more curved) by providing more temporal context, it also significantly increases computational load. As a result, the system may struggle to keep up with the camera stream, leading to latency. The goal is to determine the optimal number of frames that should be fed into neural networks.

3.1.1. Impact of Frame Skip on Performance and Accuracy

The experiments were conducted in a *Jupyter Notebook*. Specifically, it contains six experiments with different frame skip values. The frame skip parameter determines how many frames are skipped before passing the next one to the neural network. Then, it captures frames from the *video.mp4* (created for experiment). The video was recorded in *FHD*, *60 FPS*. For each experiment, several metrics were calculated and recorded, including the number of trajectory points, the average *FPS* (which varied due to the computational load of the neural network), and the total processing time of the frames fed to neural network. The metrics are saved in the */results* folder as markdown table. The table below (see table 3) demonstrates that as the *Frame Skip* value increases, the number of processed frames and number of trajectory points decreases, while the average FPS improves significantly and the overall processing time is reduced.

table 3

Metrics for Different Frame Skip

Frame Skip	Frames Fed	Trajectory Points	Avg. FPS	Processing Time (s)
0	60	423	20.92	20.016
1	30	210	33.37	9.328
2	20	140	42.69	6.656
3	15	103	47.98	4.844
4	12	79	50.85	3.891
5	10	61	52.87	3.062

The picture below (see Fig. 25) illustrates the plots created using *matplotlib* and *seaborn* libraries. Saved in */results* folder named as *plot.png*. As seen in the left plot, increasing the frame skip significantly improves the average *FPS*. However, it is obvious that “*Frames Fed*” decreases non-linearly. The middle plot shows that the total processing time decreases sharply as fewer frames are passed to the neural network. However, as demonstrated in the right plot, this comes at the cost of trajectory points, meaning less curve of the signature becomes less detailed.

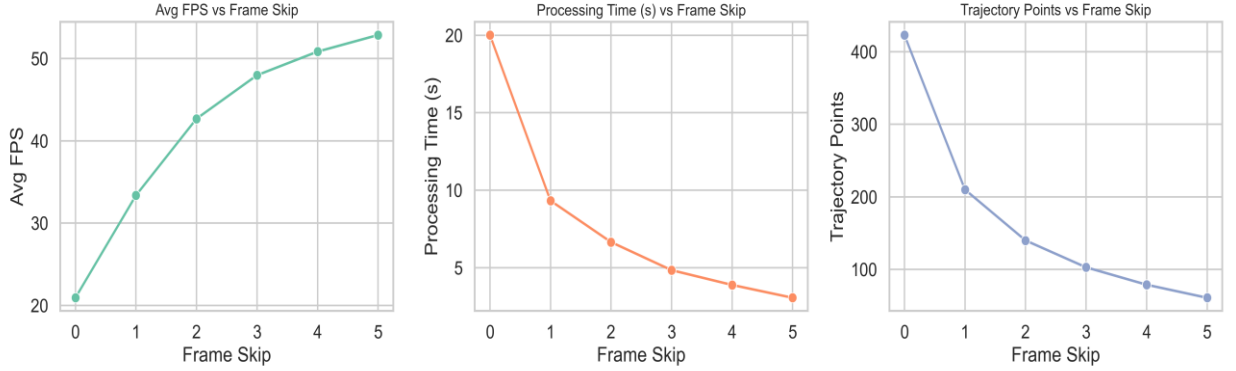


Fig. 25. Frame skip vs other metrics

3.1.2. Trajectory Visualization

The additional step is to show the trajectory plots of coordinates captured from the hand. The picture below (see Fig. 26) shows the trajectory of finger movement captured under different *frame_skip* settings, ranging from 0 to 5. These plots show how skipping frames before passing frame to the neural network affects the detail and continuity of the captured trajectory. The *frame_skip* of 0 provides dense, smooth, and highly detailed trajectory, due to all 60 frames being used. The *frame_skip* of 1 provides slightly reduced density with good continuity, still retains most details while significantly improving processing performance. The *frame_skip* of 2 still looks similar to the original, but it is starting to show some gaps. The *frame_skip* of 3 looks more jagged, path looks less natural. The *frame_skip* of 4, now the path is quite rough, many curves are lost, and the movement looks more as a series of straight lines. The *frame_skip* of 5 is the most simplified version, the trajectory is very blocky, and it misses a plethora of original motion.

As expected previously, when *frame_skip* increases, the number of frames processed decreases, leading to faster computation and higher *FPS* of the system, but quality of trajectory reduces. This figure clearly demonstrates the trade-off between accuracy and smoothness of the system. Since, most of the cameras are 30 FPS and it is a common cap in operating systems, it is reasonable to choose *frame_skip* of 1, it is a great compromise between smoothness and details of the finger motion. However, for low-end computers, a *frame_skip* of 2 can be a better choice.

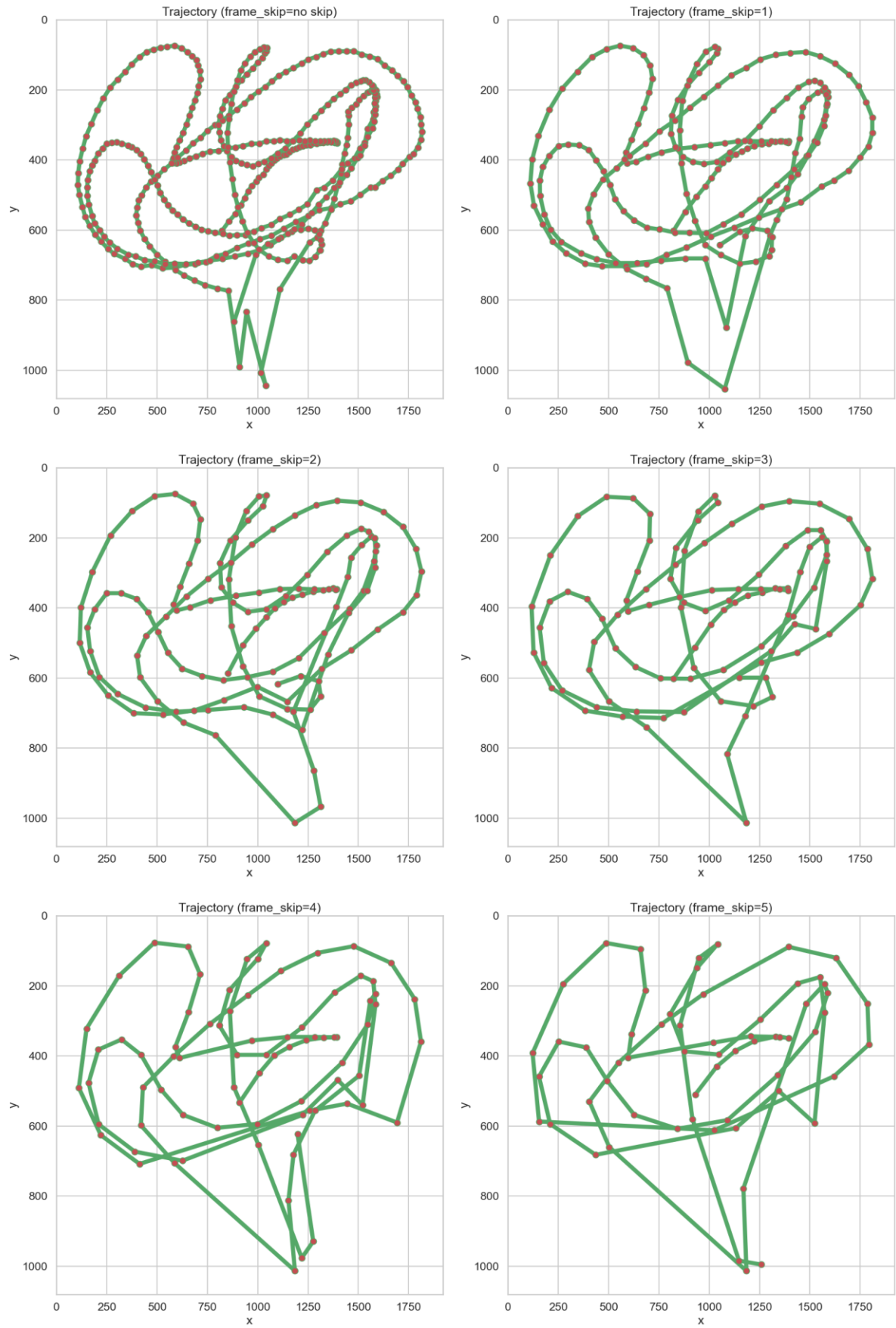


Fig. 26. Trajectory for each frame skip

3.2. Background Removal

One of the key challenges in hand gesture recognition and signature trajectory tracking is the influence of the background. In conditions of imperfect lighting, the presence of different objects, or a complex background, the system may have difficulty to accurately localize the hand. It may lead to reduced accuracy of recognition, false activations and unstable movement trajectory.

3.2.1. Algorithm for Background Removal

The following algorithm describes the extraction of the foreground based on the depth map. This approach assumes that a depth estimation model is applied to each frame of the input video stream, which predicts a depth map $D(x, y)$ of size $H \times W$, where each value corresponds to the depth of a pixel. The smaller the depth value, the closer the object is to the camera. To select the foreground, a depth threshold T is introduced. All pixels which the depth is less than or equal to this threshold are considered as a part of foreground. Based on this, a *binary mask* $M(x, y)$ is calculated. This mask can be applied to original images to nullify pixels of background.

Algorithm: Background Removal

Input: Estimated depth map $D(x, y) \in \mathbb{R}^{H \times W}$, image $I(x, y) \in \mathbb{R}^{H \times W}$ and depth threshold T .

Output: Image of extracted foreground $I_{fg}(x, y) \in \mathbb{R}^{H \times W}$.

1. Define a binary mask $M(x, y) \in \mathbb{R}^{H \times W}$ that identifies pixels as follows:

$$M(x, y) = \begin{cases} 1, & \text{if } D(x, y) \leq T \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

2. Apply obtained mask to the original *RGB* image $I(x, y)$ to find $I_{fg}(x, y)$:

$$I_{fg}(x, y) = M(x, y) \cdot I(x, y) \quad (4)$$

3. Alternatively, the background can be replaced with a solid colour C resulting in:

$$I_{fg}(x, y) = M(x, y) \cdot I(x, y) + (1 - M(x, y)) \cdot C \quad (5)$$

4. Return I_{fg} and end the algorithm.
-

Consequently, this method enables the extraction of clean foreground images, serving as a preprocessing step prior to feeding the image into a neural network. This can be especially useful in tasks such as hand tracking, where removing the background helps the model focus on the relevant features. It also reduces noise and makes the input more consistent.

3.2.2. Applying *MiDaS*

The use of the *MiDaS* depth estimation is considered as a potential solution. *MiDaS* allows to get depth maps of the image, estimating the relative distance from camera to objects. Thus, the nearest objects can be selected (e.g., hand, person), separating them from the background. In the image below (see Fig. 27), three types of depth models are provided, ranging from a weak model to a strong one. Therefore, it is evident that the bright blue area can be extracted from the image and passed as input to the neural network.

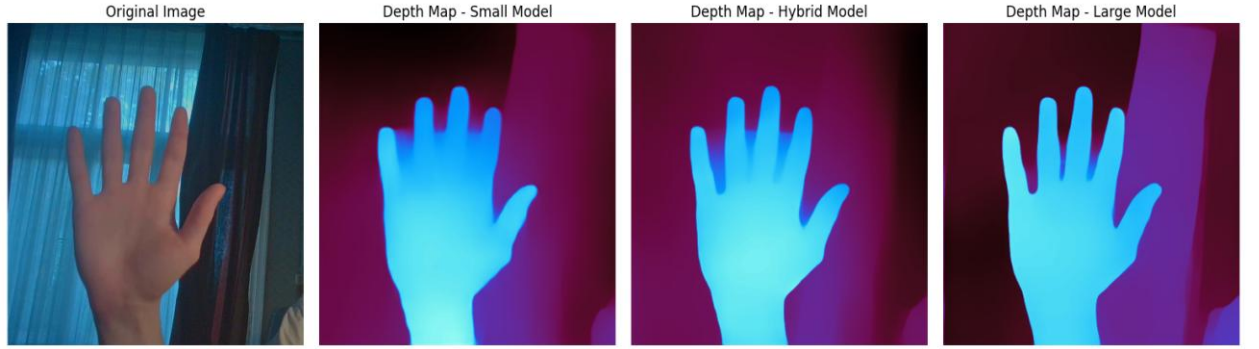


Fig. 27. Depth map of three models

The implementation of background removal based on the depth is the following: after capturing a frame, the depth map is predicted by the *MiDaS* model. The next step is to apply depth threshold filtering (see Fig. 28) – pixels located at a distance closer than a certain value are saved, but the others are nullified or replaced by the solid background. It allows to get the mask of a foreground, which contains the hand and fed it to the neural network for hand detection.

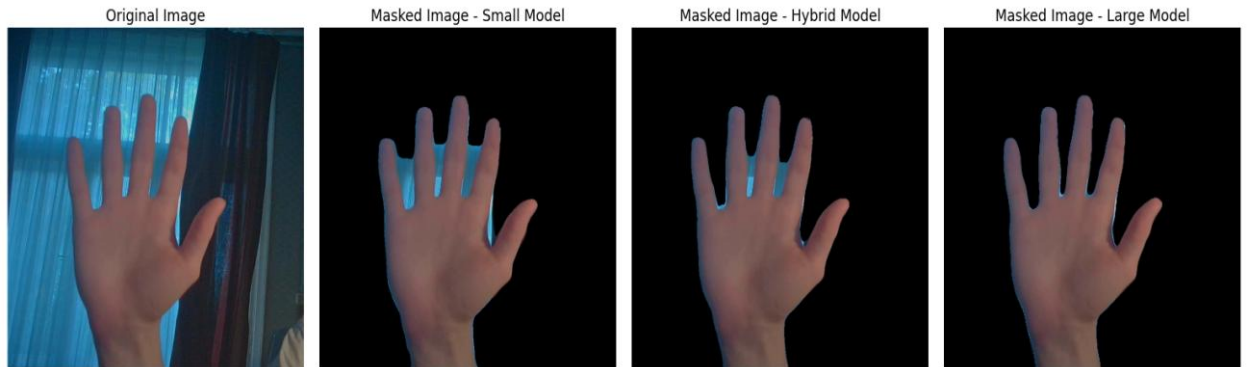


Fig. 28. Background removal based on depth threshold

Although an integration of *MiDaS* into the main pipeline was not implemented within this work due to performance and time constraints. The conducted experiments approved the perspectives of such approach, but on a modern *GPU*, because the depth estimation model is quite resource consuming. When running on devices without a *GPU* (e.g., laptops, regular *PCs*), performance drops significantly, and the system cannot maintain the required *FPS* for real-time video.

As shown in a table (see table 4), the performance of the model depends on the model itself and used device. On a *CPU*, even the fastest model (*MiDaS Small*) processes a frame in approximately 169 milliseconds, while on a *GPU*, the same frame is processed in only 31 milliseconds (~5.45 times faster). More advanced models such as *DPT Hybrid* and *DPT Large* require more time on a *CPU* (up to 3.5 seconds). Therefore, it is not reasonable to use large, even small ones without a graphical accelerator, since delay is noticeable. Additionally, simpler models may struggle when faced with complex or highly varied backgrounds, which can negatively impact the user experience. Because of this, it is expected that more advanced models will be used in practical scenarios. A prime example is the *DepthAnything* model (see Fig. 5), which has shown impressive results even with distorted or low-quality images. Ultimately, there are perspectives of using depth estimation neural networks for background removal in application of remote signing, but it should be done either on the powerful *GPU* or through *SaaS* (Software as a Service) infrastructure.

table 4

MiDaS Inference Time on i7-6700 and GTX 1070

Model	Avg. Processing Time (CPU)	Avg. Processing Time (GPU)
<i>MiDaS Small</i>	169 ms	31 ms
<i>MiDaS DPT Hybrid</i>	1640 ms	100 ms
<i>MiDaS DPT Large</i>	3513 ms	155 ms

CONCLUSION

The objective of this work was to develop a real-time remote signing system using a single camera, allowing users to create digital signatures remotely. The importance of such a system arose due to the pandemic, which boosted remote technologies. Moreover, it saves a significant amount of time, as it eliminates the need for physical presence and allows all tasks to be completed from home. The project successfully completed several core tasks, including real-time hand detection and tracking, gesture recognition, and depth estimation. Using a single camera makes the system more affordable and accessible since most devices already have a built-in webcam. This approach also does not require any other types of peripherals or equipment. Despite using only one camera, the system can still achieve previously described goal.

To reach this goal, an overview of computer vision methods and technologies was performed, exploring the basics of computer vision and the *OpenCV*. For depth estimation, different depth estimation models were explored, including *MiDaS*, *MonoViT*, and *DepthAnything*. For hand tracking, libraries such as *OpenPose* and *MediaPipe* were considered. *MediaPipe* was selected due to its real-time performance and lightweight architecture, moreover, it was compatible with gesture recognition and provided the minimal depth estimation (only for the hand). Furthermore, related projects were reviewed to identify existing limitations and potential areas for improvement.

The main development process began in the second part of the project. Initially, it started with system analysis to identify the functional, non-functional, user and system requirements. This stage ended by the use case diagram. Therefore, it provided comprehensive system overview. The next phase was system design that describes the algorithms, logic, relationships and classes through sequence, class, and component diagrams. This stage was crucial, as it established the foundation of the software and helped to understand the workflow of the system.

After describing the requirements and workflow of the software the implementation phase began. This phase included the creation of a wireframe, which helped to understand the structure and functionality of the user interface. Then, the final graphical user interface was done using *PySide 6*. The next stage of implementation consisted of building the project using *PyInstaller* and creating an installer using the *Inno Setup* tool. It provided a convenient interface for installing the required application files and ensured a simple way to launch the program. Additionally, the installer compressed the application from 300 MB to 100 MB, reducing the time of downloading.

Finally, the software was tested both manually and automatically. Manual testing was conducted from the user's perspective by performing actions such as clicking, drawing, and modifying elements. This ensured that user experience was implemented as expected and there were no user interface bugs. As part of automated testing, 64 unit tests were written to cover both user actions

and the system’s internal logic. As a result, a markdown file was generated, containing the test execution date, names of the tested modules, number of tests, execution time, and test statuses. Below the table, the overall test pass rate is displayed, which was 100% for the system.

The final part of the thesis was devoted to experiments and potential approaches to improve the system. There were several problems, such as complex backgrounds, poor lighting conditions and video stream latency. The first section focuses on optimizing the video stream by analysing the impact of frame skipping prior to feeding frames into the hand detection model. Six experiments were conducted with different values of *frame_skip* parameter (from 0 to 5). Increasing *frame_skip* increases *FPS* and decreases processing time but reduces the detail of the signature trajectory. Moreover, the trajectories of the finger movement were visualised for each *frame_skip* value. As a compromise, *frame_skip* of 1 was recommended – it gives a good balance between accuracy and performance. For low-end computers, a *frame_skip* value of 2 can be used.

The next section was dedicated to approach of removing background using depth estimation neural networks. This method was considered as preprocessing step of frame before passing it to hand detection model. The section is followed by the mathematical algorithm to show the steps such as binary mask creation, thresholding, and replacement by solid colour. Then the algorithm was practically applied to the frames in *Jupyter Notebook*, and it successfully removed the background, extracting the hand. Finally, the algorithm was tested for 30 frames to estimate the processing time both on *CPU* and *GPU*. As a result, the *GPU* was significantly faster than *CPU*.

Despite the successful implementation, the system faced several limitations. These limitations are not significant but can be improved in the future. The most notable issues include:

- *Dependency on lighting conditions.* Poor lighting can negatively affect hand detection. Especially, the case of direct sunlight or other sources of light, blurring the edges of the hand.
- *Limited set of gestures.* Currently, the system recognizes only a small number of gestures. It is suggested to add more gestures, for instance, gestures for enabling “clearing mode” would allow users to correct parts of the signature without erasing it entirely.
- *Performance without GPU.* Depth estimation models like *MiDaS* are resource consuming and performs slowly on systems without *GPU*. It is suggested to implement the application as a *SaaS* and explore more efficient models like *DepthAnything*, which may offer improved accuracy and better performance.

However, these limitations do not critically impact the system’s core functionality but highlights areas for further improvement and optimization. By fixing this limitation, the system can become faster and more accurate. In addition, future work may include integration with other platforms, it will make the system better for real-world applications and more reliable for users.

REFERENCE LIST

1. Cao, Z. *et al.* (2017) “Realtime multi-person 2D pose estimation using part affinity fields,” in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE.
2. Comaniciu, D. and Meer, P. (2002) “Mean shift: a robust approach toward feature space analysis,” *IEEE transactions on pattern analysis and machine intelligence*, 24(5), pp. 603–619. Available at: <https://doi.org/10.1109/34.1000236>.
3. *Depth Anything* (2024) *Github.io*. Available at: <https://depth-anything.github.io>.
4. Garrett, J.J. (2010) *The elements of user experience: User-centered design for the web and beyond*. 2nd ed. Upper Saddle River, NJ: New Riders Publishing.
5. Gonzalez, R.C. and Woods, R.E. (2017) *Digital Image Processing*. 4th ed. Upper Saddle River, NJ: Pearson.
6. *Hand landmarks detection guide* (2025) *Google AI for Developers*. Available at: https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker.
7. He, K. *et al.* (2015) “Deep residual learning for image recognition,” *arXiv [cs.CV]*. Available at: <http://arxiv.org/abs/1512.03385>.
8. Mascarelli, A. *et al* (2023) *Identification of Finger-based Signatures Using Computer Vision Techniques*. Available at: https://github.com/FallenElias/Finger_AirSigning_Project.
9. Oquab, M. *et al.* (2023) “DINOv2: Learning robust visual features without supervision,” *arXiv [cs.CV]*. Available at: <http://arxiv.org/abs/2304.07193>.
10. Ranftl, R. *et al.* (2022) “Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer,” *IEEE transactions on pattern analysis and machine intelligence*, 44(3), pp. 1623–1637. Available at: <https://doi.org/10.1109/TPAMI.2020.3019967>.
11. Shapiro, N. and Malinowski, M. (2023) *MiDaS*, *Github.com*. Available at: <https://github.com/isl-org/MiDaS>.
12. Shotton, J. *et al.* (2011) “Real-time human pose recognition in parts from single depth images,” in *CVPR 2011*. IEEE.
13. Sommerville, I. (2015) *Software Engineering, Global Edition*. 10th ed. London, England: Pearson Education.
14. Victor, B. *et al* (2024) *Remote Scan Recognition*, *Github.com*. Available at: <https://github.com/AllouetteVB/RemoteScan-Recognition>.
15. Wachs, J.P. *et al.* (2011) “Vision-based hand-gesture applications,” *Communications of the ACM*, 54(2), pp. 60–71. Available at: <https://doi.org/10.1145/1897816.1897838>.
16. Yan, J. *et al.* (2021) “Channel-wise attention-based network for self-supervised monocular depth estimation,” *arXiv [cs.CV]*. Available at: <http://arxiv.org/abs/2112.13047>.

17. Yang, L. *et al.* (2024) “Depth Anything: Unleashing the power of large-scale unlabeled data,” *arXiv [cs.CV]*. Available at: <http://arxiv.org/abs/2401.10891>.
18. Zhang, F. *et al.* (2020) “MediaPipe Hands: On-device real-time hand tracking,” *arXiv [cs.CV]*. Available at: <http://arxiv.org/abs/2006.10214>.
19. Zhao, C. *et al.* (2022) “MonoViT: Self-supervised monocular depth estimation with a vision Transformer,” *arXiv [cs.CV]*. Available at: <http://arxiv.org/abs/2208.03543>.
20. Zhaocq (2022) *MonoViT: Self-supervised monocular depth estimation with a vision transformer*. Available at: <https://github.com/zxcqlf/MonoViT>.
21. Zhou, H., Greenwood, D. and Taylor, S. (2021) “Self-supervised monocular depth estimation with internal feature fusion,” *arXiv [cs.CV]*. Available at: <http://arxiv.org/abs/2110.09482>.

APPENDICES

GitHub Repository

This repository – <https://github.com/Dast3X/Virtual-Signature> – serves as the root of the project, where the main development and core application logic were implemented. The project includes an experimental part located in the `/ipynb` folder, which contains *Jupyter* notebooks with research and experiments for background removal and frame skip.

To ensure the reliability and robustness of the software, an automated testing is provided in the `/tests` directory. This set of tests covers different modules and functionalities, then generates the `test_results.md` – table of testing results.

For convenience and ease of deployment, pre-built versions of the project along with installers are published and maintained in the Releases section of the repository. Here can be found both build package and installation: <https://github.com/Dast3X/Virtual-Signature/releases>.